DRUG REPURPOSING PLATFORM

Application Flow & Configuration Document

***Document Author**: DRP Frontend Team*
***Document Owner**: TBD*

Document Version 1.0

Document History

Document Location

This is a snapshot of the application as built on 2026-10-14 (UTC) from the DRP-UI repository, branch `main`, commit `e571db7`. Screenshots were taken the same day against the live backend. Refer to the author if you are in any doubt about the currency of this document.

Revision History

Date of this revision: 2026-10-14	Date of next revision: TBD

Revision Number	Revision Date	Summary of Changes	Changes marked
1.0	2026-10-14	Initial documentation: configuration and application flow with screenshots	

Approvals

This document requires the following approvals.

Name	Title

Distribution

This document has been distributed to

Name	Title

Glossary

Term	Full Form
DRP	Drug Repurposing Platform
TxKG	Target Identification Agent (therapeutic knowledge graph)
LitMineX	Literature Mining Agent
CurateX	Drug Curation Agent
ScreenSuite	Virtual Screening Agent (molecular docking)
NovSearch	Novelty Search Agent (patents)
UniProt	Universal Protein Resource (protein identifiers)
PDB / RCSB	Protein Data Bank / Research Collaboratory for Structural Bioinformatics
ProLIF	Protein-Ligand Interaction Fingerprints (interaction profiling library)
Mol*	Molstar, the 3D molecular viewer embedded in ScreenSuite
PMC	PubMed Central
LLR / QED / TPSA	Log-likelihood ratio / Quantitative estimate of drug-likeness / Topological polar surface area
Cognito	Amazon Cognito, the sign-in service
JWT	JSON Web Token (the ID token sent with every API call)
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
UTC	Coordinated Universal Time (all times in the app are UTC)

Contents

1. Introduction

- 1.1 Overview
- 1.2 References
- 1.3 Intended Audience

2. Application Configuration

- 2.1 Runtime and tooling
- 2.2 Environment variables (.env)
- 2.3 Sign-in (authentication)
- 2.4 API client
- 2.5 Local development and CORS
- 2.6 Long-running work (jobs)
- 2.7 Browser-side storage

3. Application Flow

- 3.1 Sign in
- 3.2 Sign up and password reset
- 3.3 Home
- 3.4 Projects
- 3.5 Research workspace
- 3.6 Step 1 · TxKG: target identification
- 3.7 Step 2 · LitMineX: literature mining
- 3.8 Step 3 · CurateX: drug curation
- 3.9 Step 4 · ScreenSuite: virtual screening
- 3.10 Step 5 · NovSearch: novelty search
- 3.11 Branch, rerun and export
- 3.12 Artifacts and lineage
- 3.13 Share
- 3.14 Recent sessions and project page

4. Current Limitations

1. Introduction

This document describes how the **Drug Repurposing Platform (DRP)** web application is configured and how a researcher moves through it, screen by screen, with screenshots of the running application.

DRP helps find existing drugs that could treat a new disease. A researcher asks a question; five AI agents then work through it one step at a time (TxKG → LitMineX → CurateX → ScreenSuite → NovSearch), and the researcher confirms each step before the next starts.

1.1 Overview

This document:

- Provides a single place to find how the frontend is set up and configured
- Explains each screen: its purpose, what the user does, what the application does and what comes next
- Records the backend calls behind each step, for developers and testers
- Gives new team members a reference to the application as built
- Lists the current limitations, so they are not reported again as new issues

1.2 References

- DRP-UI source code (React frontend), branch `main`
- DRP_Documentation.pdf (2026-10-01), rootcause.pdf and frontendrootcause.pdf (2026-10-14)
- API_INTEGRATION.md in the repository

1.3 Intended Audience

- **Developers:** to set up, run and change the application, and to find the code and endpoints behind each screen
- **Testers:** to know the expected flow and behaviour of each screen
- **Product and project stakeholders:** to see what the application does end to end

The walkthrough in section 3 is one real session: "Find protein targets for thrombocytosis", filed under the project "Thrombocytosis repurposing", taken through all five agents.

2. Application Configuration

Configuration decisions are grouped by topic, one table each.

2.1 Runtime and tooling

Subject Area	**Runtime and tooling**	**Topic**	Configuration
Setting	React 19 single-page app built with Create React App 5 (`react-scripts`); MUI 6 for UI; react-router 6; axios for HTTP; AWS Amplify 6 for sign-in; d3 / react-force-graph-2d for the knowledge graph; Mol* 4.5.0 (loaded on demand from cdn.jsdelivr.net) for 3D structures.	**Config ID**	C01
Requirement	Node.js 20 (v20.20.2 used for this document) and npm.		
Commands	`npm install` once; `npm start` runs the development server on http://localhost:3000; `npm run build` produces the production bundle in `build/`.		
Notes	`npm test` does not start yet: `@testing-library/jest-dom` is missing from the dependencies.		

2.2 Environment variables (.env)

Subject Area	**Environment variables (.env)**	**Topic**	Configuration
Setting	`REACT_APP_API_URL`: base URL of the backend API Gateway (required; the app stops on load without it). `REACT_APP_COGNITO_USER_POOL_ID`, `REACT_APP_COGNITO_APP_CLIENT_ID`: the Cognito user pool and app client. `REACT_APP_COGNITO_DOMAIN`, `REACT_APP_REDIRECT_SIGN_IN`, `REACT_APP_REDIRECT_SIGN_OUT`: hosted-UI settings (passed to Amplify, not used at runtime). `REACT_APP_COGNITO_REGION`: defined, not read.	**Config ID**	C02
Where	`src/apiconfig.js` (API URL), `src/components/Amplifyconfig.jsx` (Cognito).		
Notes	Values are environment-specific and are not reproduced here. CRA reads `.env` only when `npm start` or `npm run build` starts, so restart after changing it.		

2.3 Sign-in (authentication)

Subject Area	Sign-in (authentication)	Topic	Configuration
Setting	Email and password sign-in against Amazon Cognito through AWS Amplify (`signIn`, `confirmSignIn` for a first-time new password, `signUp` + emailed code, `resetPassword`).	**Config ID**	C03
Where	`src/components/Login/*`, `src/components/Auth/RequireAuth.jsx` (protects every `/dashboard…` route), `src/context/CurrentUserContext.js` (`GET /users/me`).		
Behaviour	After sign-in the user sees the splash screen, then Home. Tokens are refreshed by Amplify. A 401 from the backend signs the user out and returns to the sign-in page.		

2.4 API client

Subject Area	API client	Topic	Configuration
Setting	One axios instance (`src/services/apiClient.js`) for every backend call. Before each request it reads the Cognito session and sends `Authorization: Bearer <ID token>`.	**Config ID**	C04
Errors	Every error carries a readable `userMessage`, taken from the backend's `message` / `error` / `detail`, or else "Unable to reach the backend".		
Where	Service files per backend area in `src/services/api/`: sessions, jobs, modules, files, txkg, litminex, curatex, screensuite, novsearch, projects, exports.		

2.5 Local development and CORS

Subject Area	Local development and CORS	Topic	Configuration
Setting	Run the development server on **port 3000**.	**Config ID**	C05
Reason	The backend's CORS policy allows only the origin `http://localhost:3000`. On any other port sign-in works (Cognito), but every data call fails with "Unable to reach the backend".		
Optional	`src/setupProxy.js` forwards `/api/*` to the API Gateway in development; it is used only if `REACT_APP_API_URL` is set to `/api`.		

2.6 Long-running work (jobs)

Subject Area	Long-running work (jobs)	Topic	Configuration
Setting	Every agent run is a backend job. The app polls `GET /agents/jobs/{jobId}/status` (first poll at once, then backing off from 1 s to 5 s), retries up to 3 network errors and polls again when the tab becomes visible.	**Config ID**	C06
Behaviour	The card shows the job's progress message and how long it has been running; after 10 minutes without change it offers "Keep waiting" or "Stop waiting". When the job completes, the module's results are loaded and the session is re-read.		
Where	`src/hooks/useJob.js`, `src/hooks/usePhaseResults.js`, `src/workflow/phaseResults.js`.		

2.7 Browser-side storage

Subject Area	Browser-side storage	Topic	Configuration
Setting	Until the backend stores them, the app remembers in the browser: a session's project, CurateX's candidates job per CurateX step, and NovSearch patent questions and answers per run.	**Config ID**	C07
Where	`src/workflow/sessionStore.js` (localStorage keys `drp.sessionProjects`, `drp.curatexCompoundsJobs`, `drp.patentQa`).		
Implication	These survive a reload or reopen in the same browser only.		

3. Application Flow

Each step of the flow is described in one table (purpose, what the user does, what the application does, what comes next), followed by its screenshots.

3.1 Sign in

Subject Area	**Sign in**	**Topic**	Account
Purpose	Authenticate the researcher.	**Flow ID**	F01
User action	Enter email and password and click **Sign In**. Links lead to Sign up, Forgot password, Help, Privacy and Terms.		
What happens	Cognito `signIn`; on success the splash screen ("Preparing your research workspace") shows for about 5 seconds, then Home.		
Next	Home (F03).		

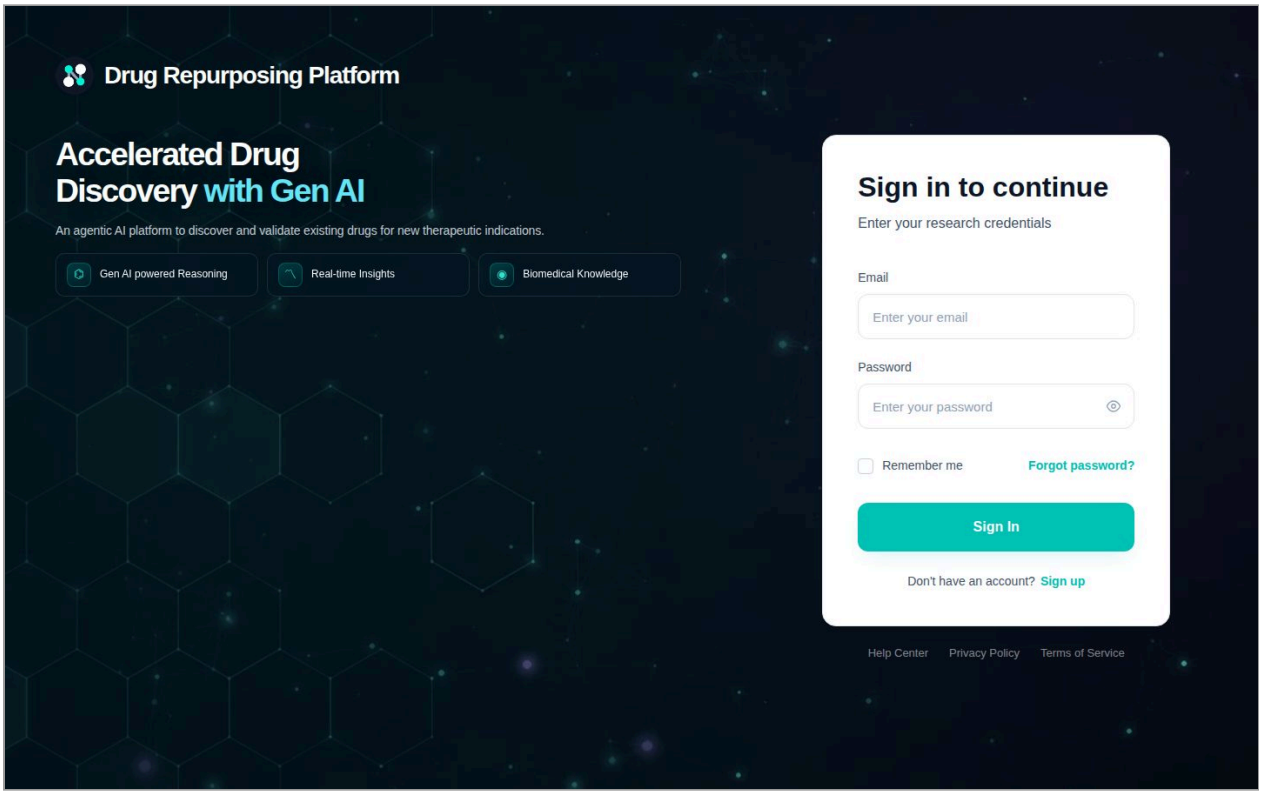


Figure 3.1.1 Sign-in screen (`/login`).

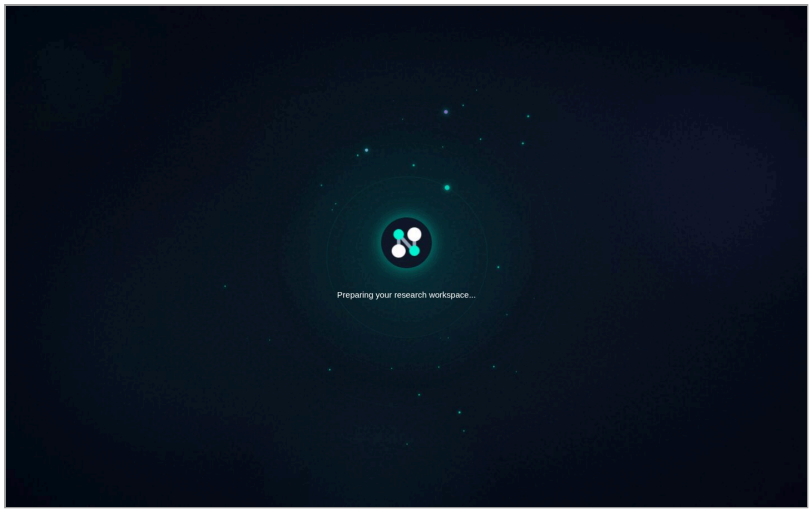


Figure 3.1.2 Splash screen after sign-in.

3.2 Sign up and password reset

Subject Area	Sign up and password reset	Topic	Account
Purpose	Create an account, or reset a forgotten password.	**Flow ID**	F02
User action	**Sign up**: name, email, password, confirm, accept terms, then the code sent by email. **Forgot password**: email, then the code and a new password.		
What happens	Cognito `signUp` / `confirmSignUp`, or `resetPassword` / `confirmResetPassword`; the user returns to Sign in with a notice.		
Notes	Help, Privacy and Terms are reachable without signing in; their final text is still to be provided.		

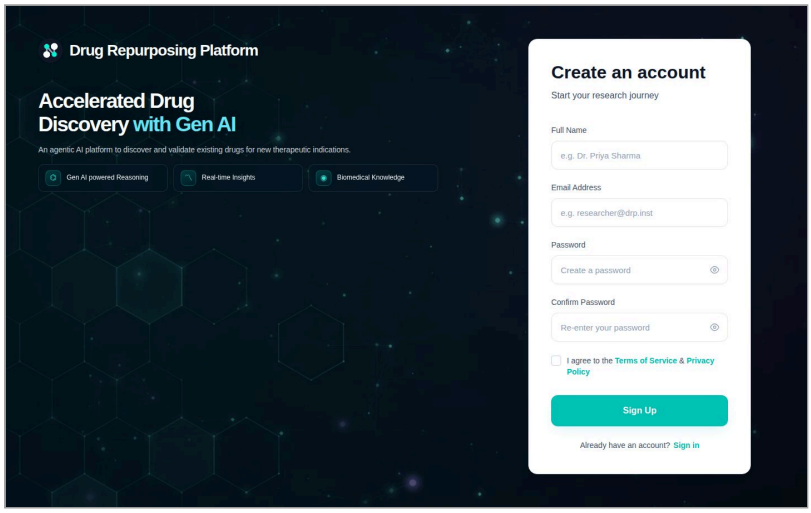


Figure 3.2.1 Create an account (`/signup`).

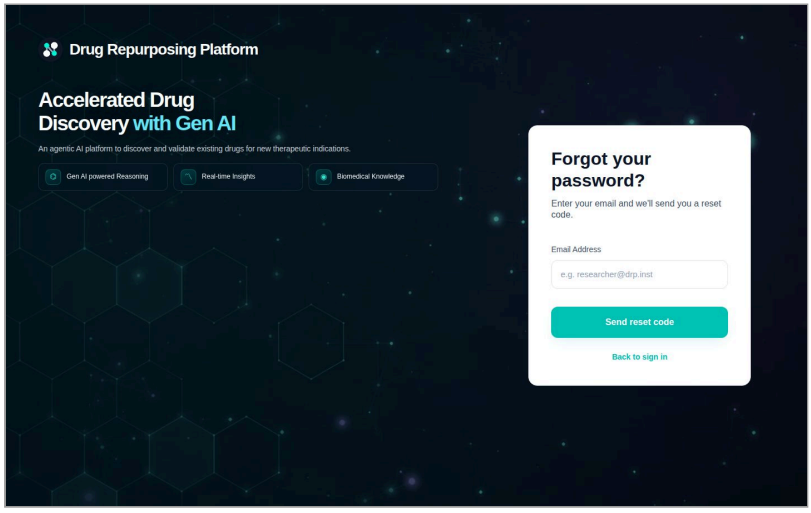


Figure 3.2.2 Forgot password (`/forgot-password`).

3.3 Home

Subject Area	Home	Topic	Start research
Purpose	Start a research session from a question.	**Flow ID**	F03
User action	Type a disease or question and click **Begin research**. The **+** button offers: upload files, **Add to project** (pick or create a project) and **Choose module** (start in a specific agent, or Auto).		
What happens	`POST /sessions { query, module, projectId }` creates the session and starts the first agent (TxKG here). Quick-start chips, the latest sessions and the at-a-glance totals come from `GET /dashboard/quick-actions`, `GET /sessions` and `GET /dashboard/summary`.		
Next	The research workspace (F05).		

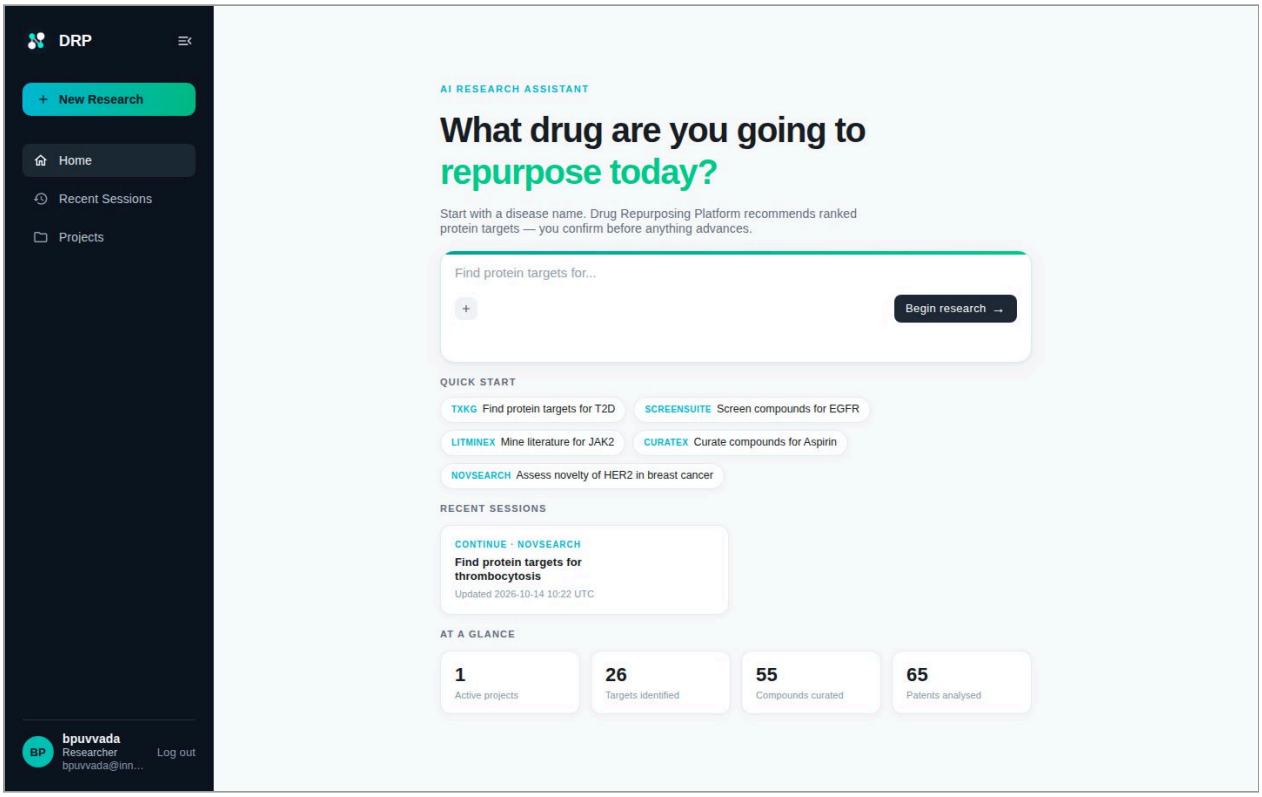


Figure 3.3.1 Home: question box, quick start, recent sessions and totals.

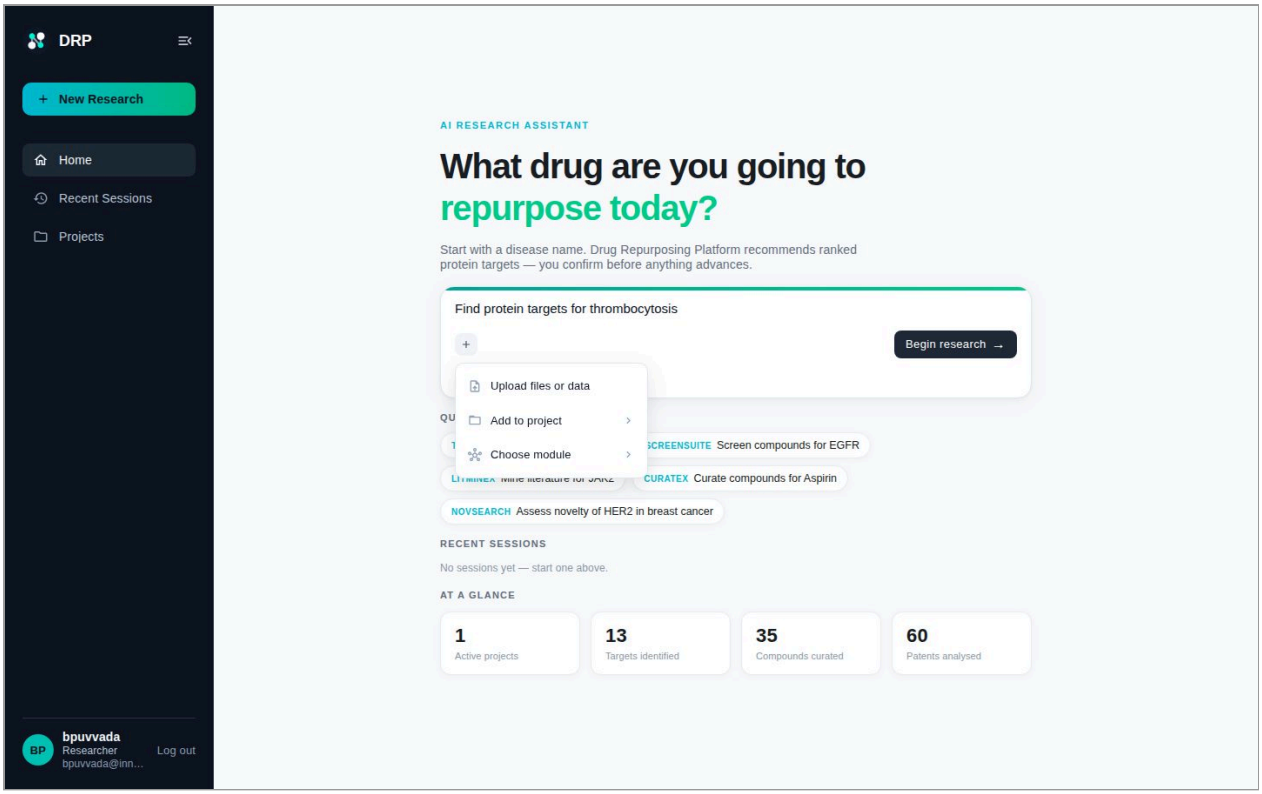


Figure 3.3.2 Home with a question typed and the + menu open.

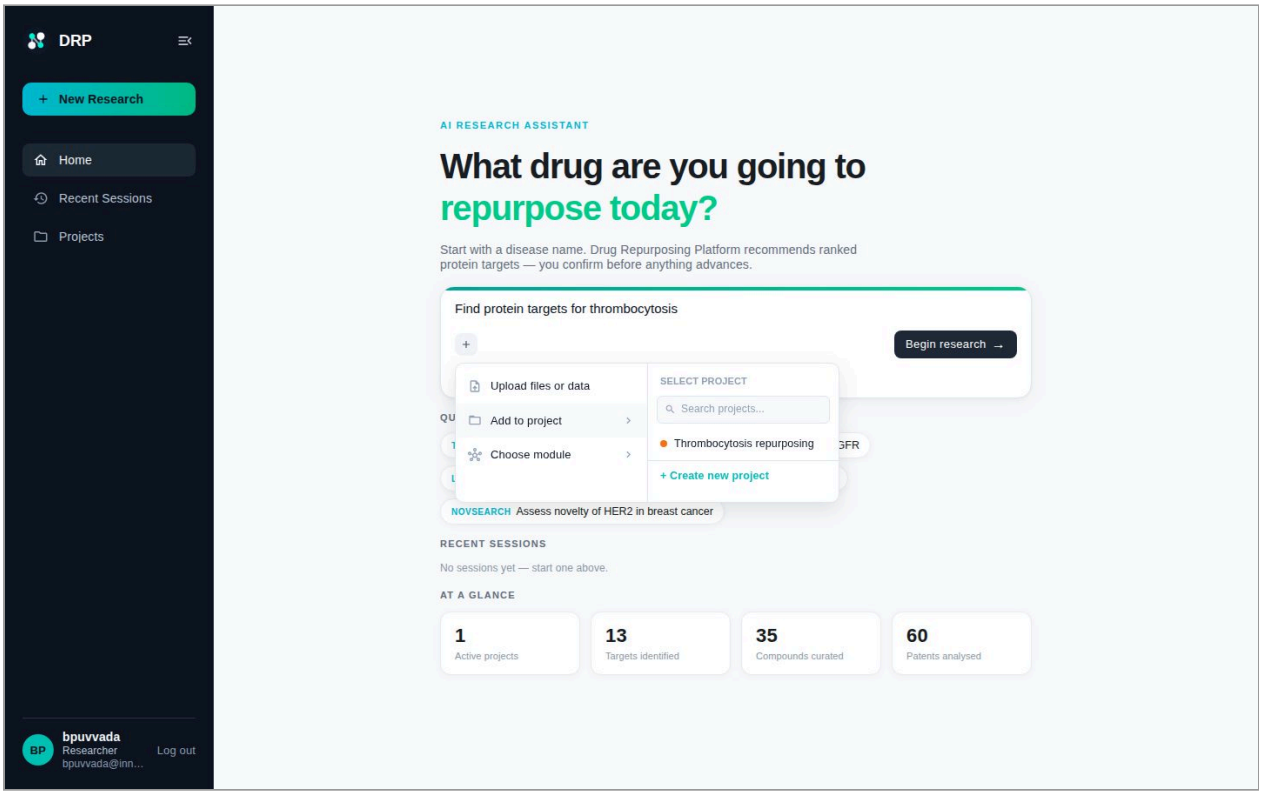


Figure 3.3.3 Add to project: choose an existing project or create one.

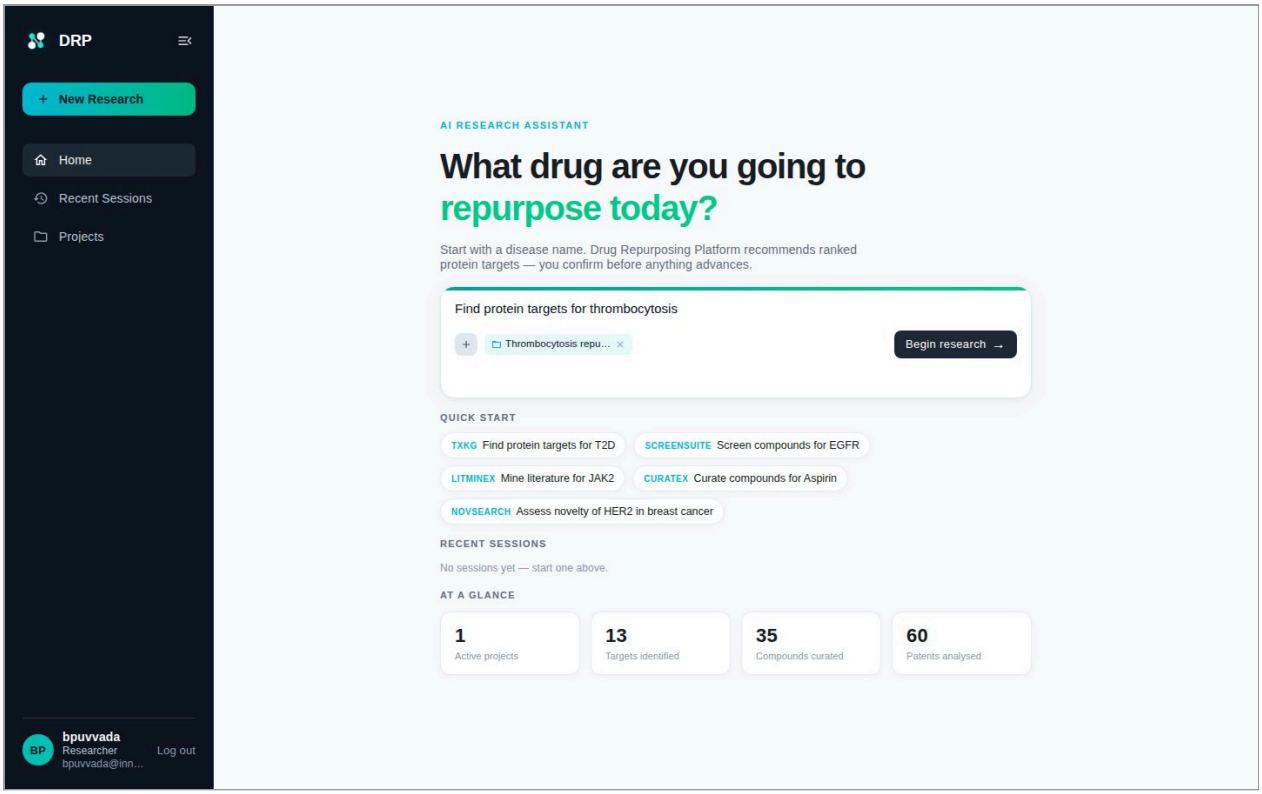


Figure 3.3.4 Ready to begin: the chosen project is shown as a chip.

3.4 Projects

Subject Area	Projects	Topic	Organise
Purpose	Group research about one disease.	**Flow ID**	F04
User action	**New Project** (name and disease); search and filter by status; rename, change status or delete from the row menu; open a project to see its sessions and saved results.		
What happens	`GET/POST/PATCH/DELETE /projects`; a project page lists `GET /projects/{id}/items` and the sessions those results came from.		

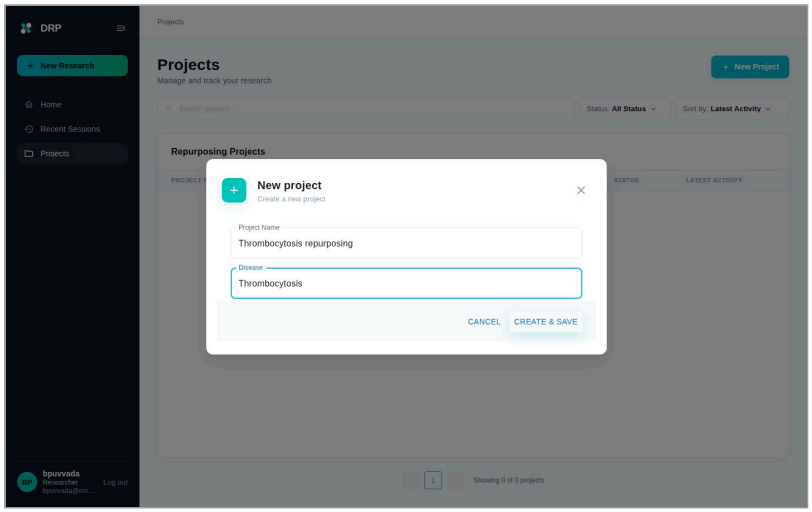


Figure 3.4.1 New project dialog.

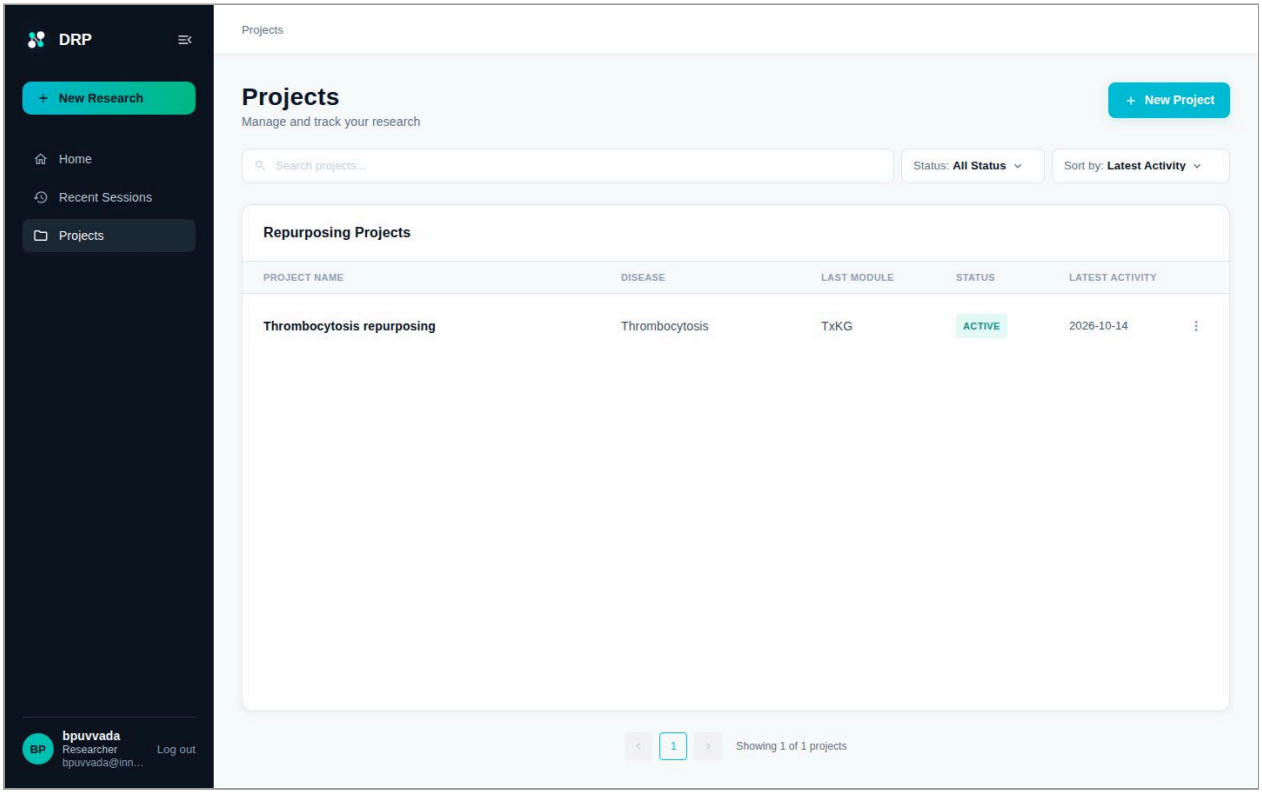


Figure 3.4.2 Projects list.

3.5 Research workspace

Subject Area	Research workspace	Topic	Session
Purpose	Where all agent work happens, as one conversation.	**Flow ID**	F05
Layout	Breadcrumb (project / disease / current agent); **Chat · Artifacts · Lineage** tabs; status pill; **BRANCH** menu; **Share**; the step rail 01 TxKG to 05 NovSearch (tick = done, red ! = failed); the timeline of messages and one card per agent run; the chat bar (type a question, or @ to start an agent).		
What happens	Each agent run is a job (C06). A finished card offers **Branch**, **Rerun** and **Export**, and a button to continue to the next agent. The session's URL (…`/workflow?session=<id>`) reopens it.		

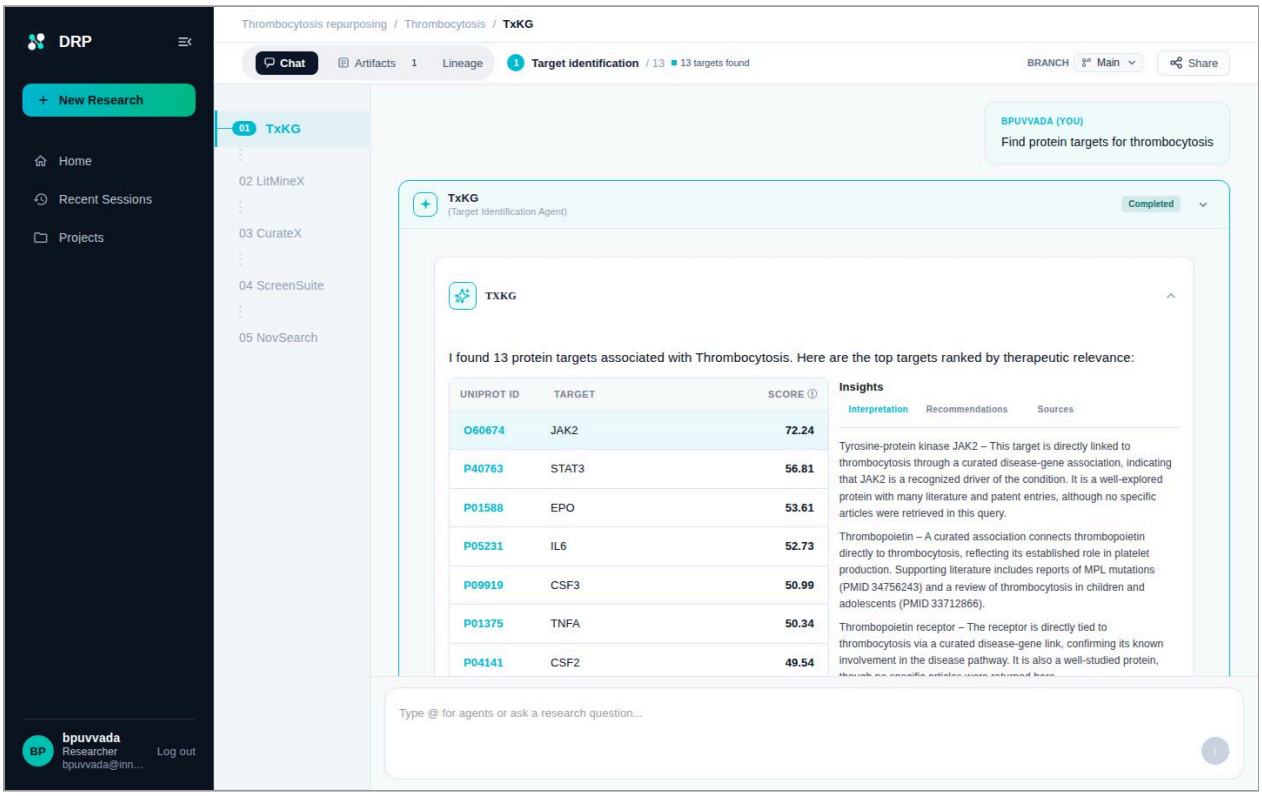


Figure 3.5.1 The workspace, showing TxKG results in a session filed under the project "Thrombocytosis repurposing".

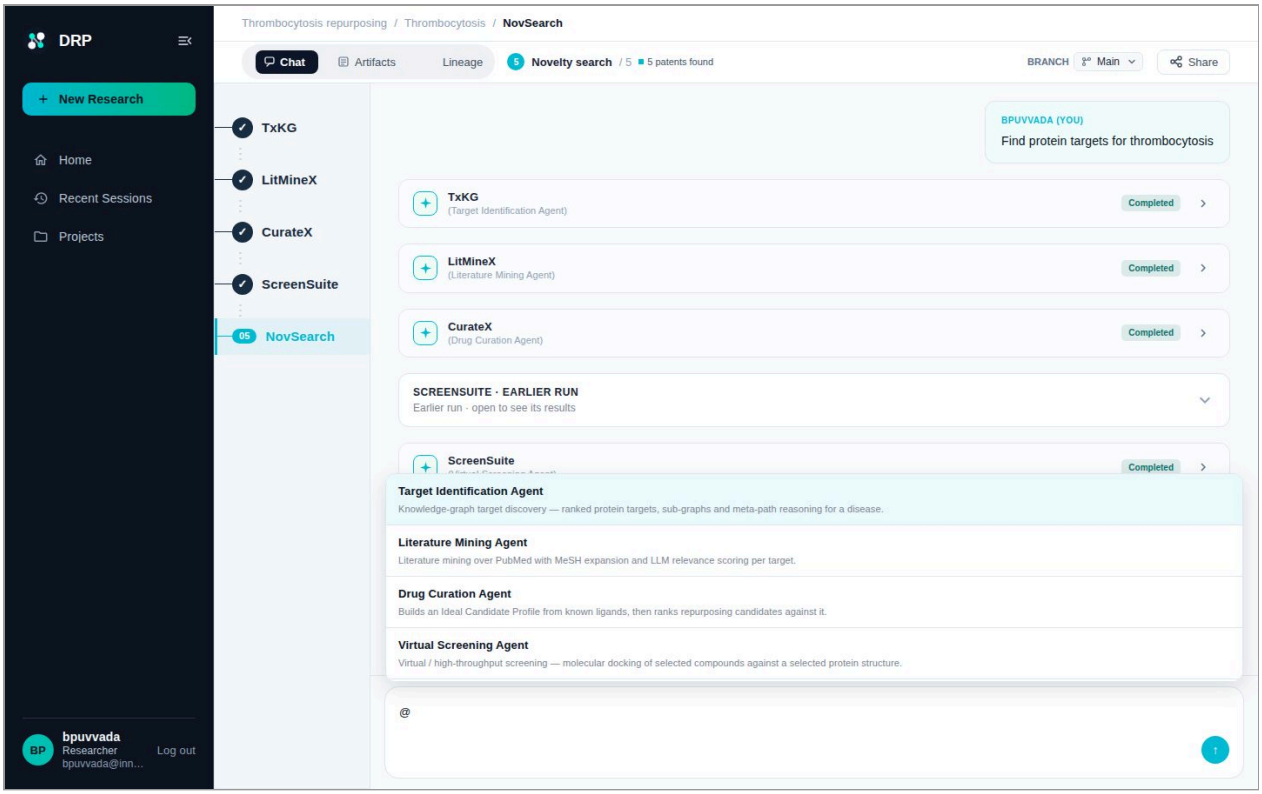


Figure 3.5.2 Typing @ in the chat bar lists the agents.

3.6 Step 1 · TxKG: target identification

Subject Area	**Step 1 · TxKG: target identification**	**Topic**	Agent
Input	The disease question from Home.	**Flow ID**	F06
What happens	The TxKG job ranks protein targets for the disease (about a minute). Results come from `GET /agents/jobs/{jobId}/result`; insights from `GET /agents/txkg/insights/{jobId}`; target details from UniProt via `GET /agents/txkg/targets/{uniprotId}`.		
Output	Ranked targets with scores (ⓘ explains the score), Insights (Interpretation, Recommendations, Sources), the knowledge graph (**Generate knowledge graph**, coloured Known / Hidden) and meta-path explanations.		
Next	**Use recommended targets**, or **Select Targets** then **Proceed to LitMinex**.		

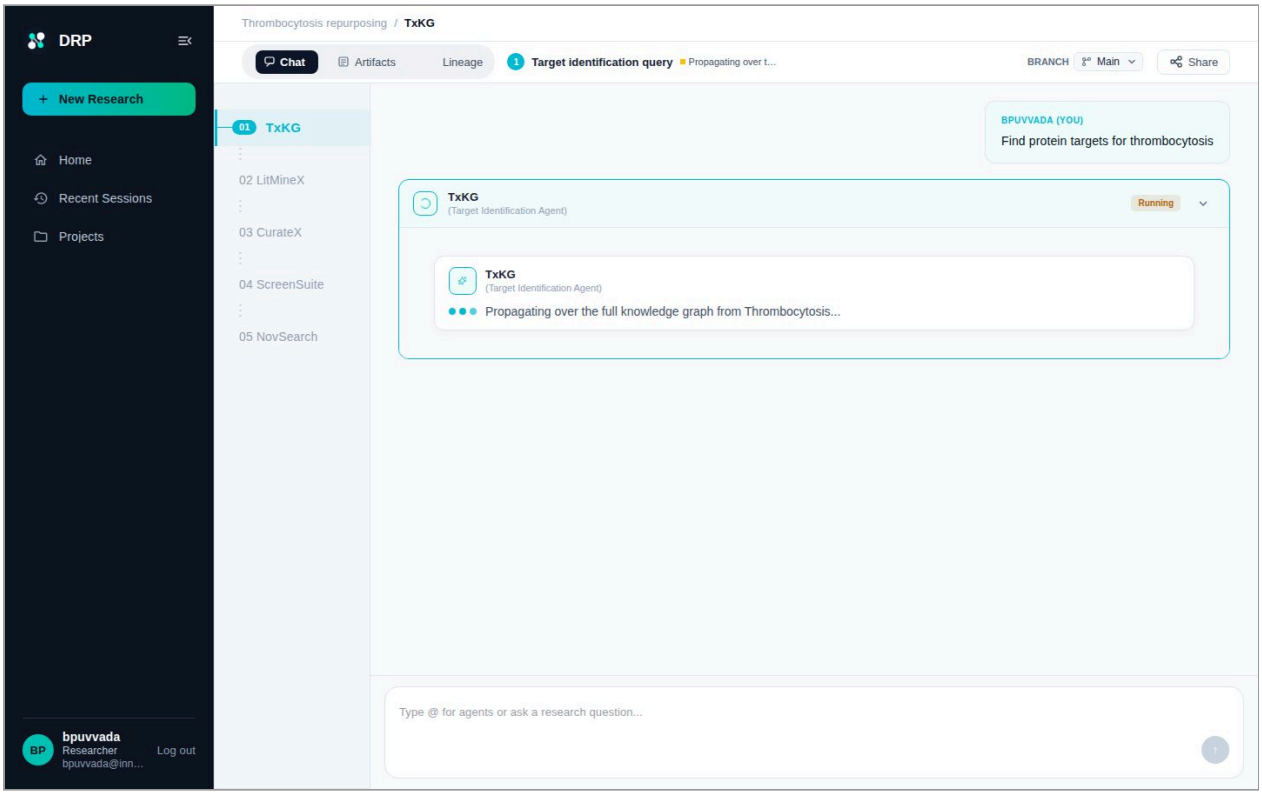


Figure 3.6.1 TxKG running, with its live progress message.

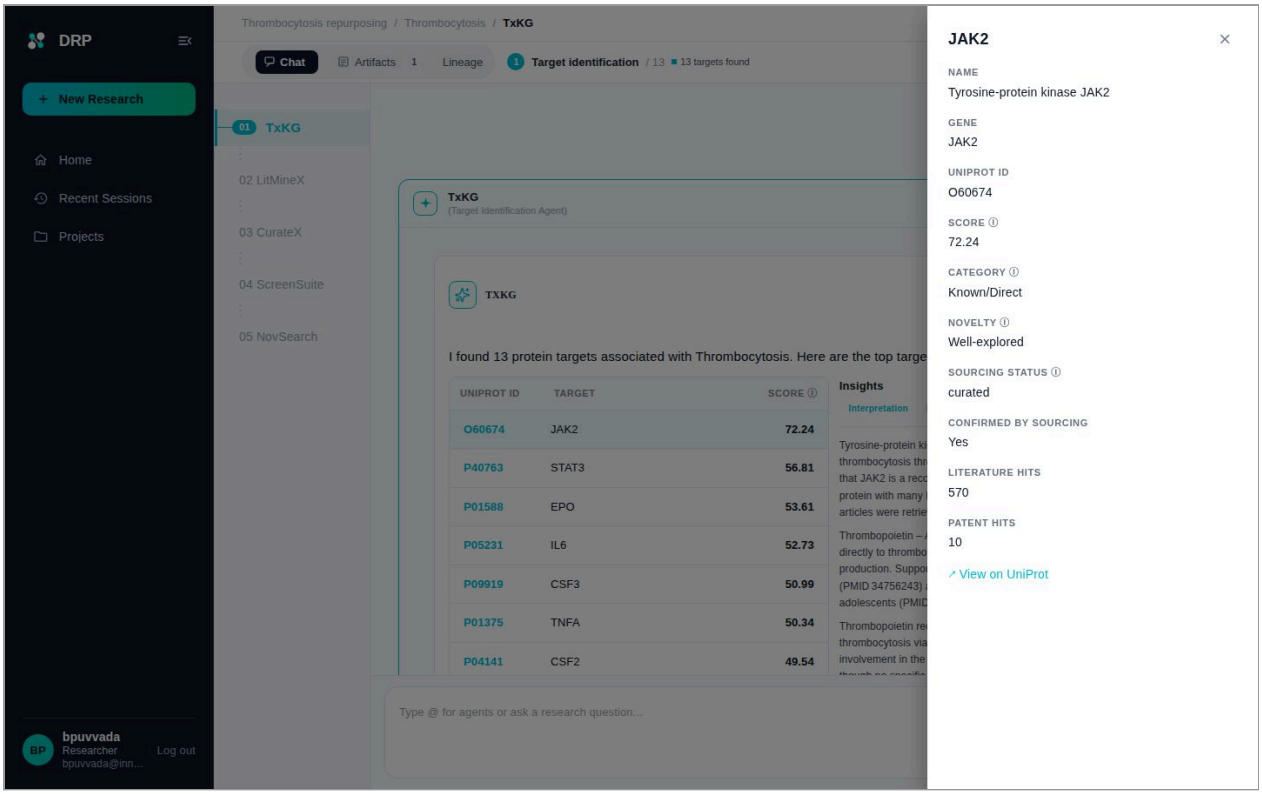


Figure 3.6.2 Target detail panel, with explanations for score, category, novelty and sourcing.

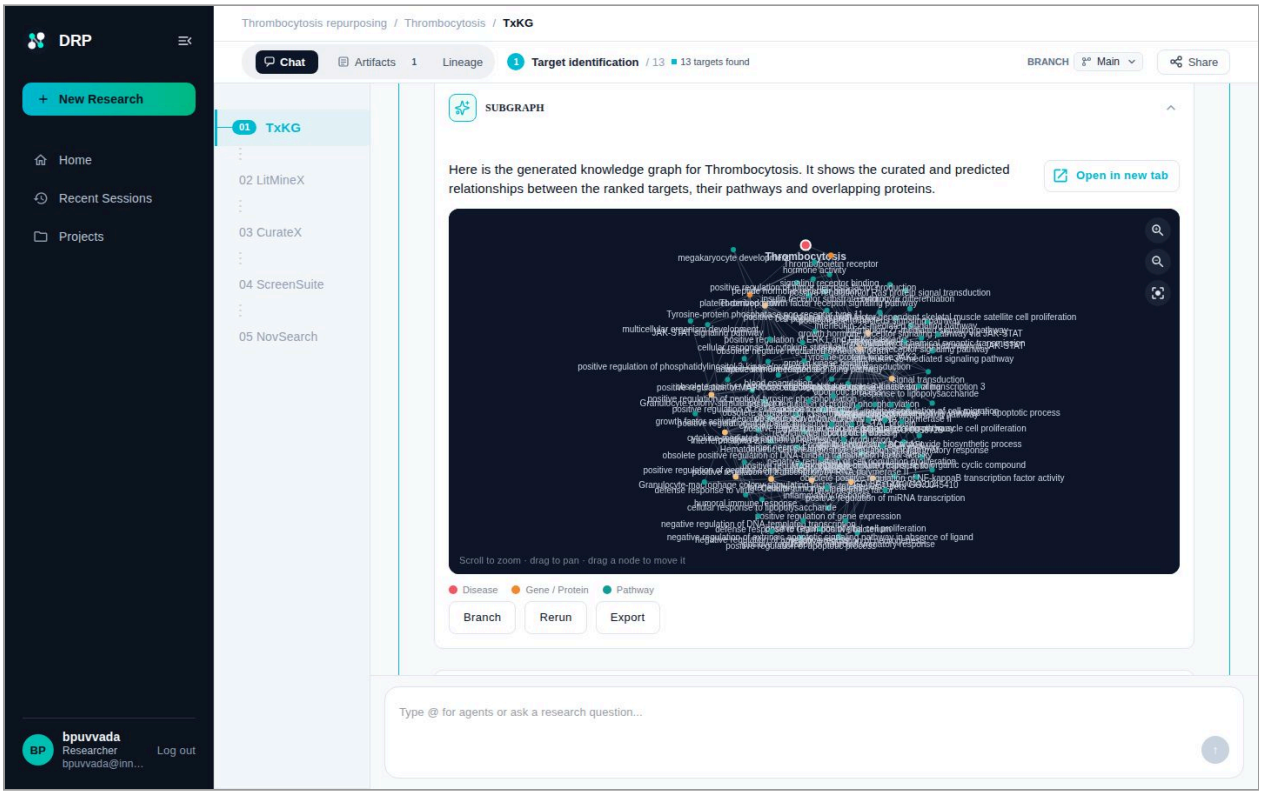


Figure 3.6.3 Generated knowledge graph.

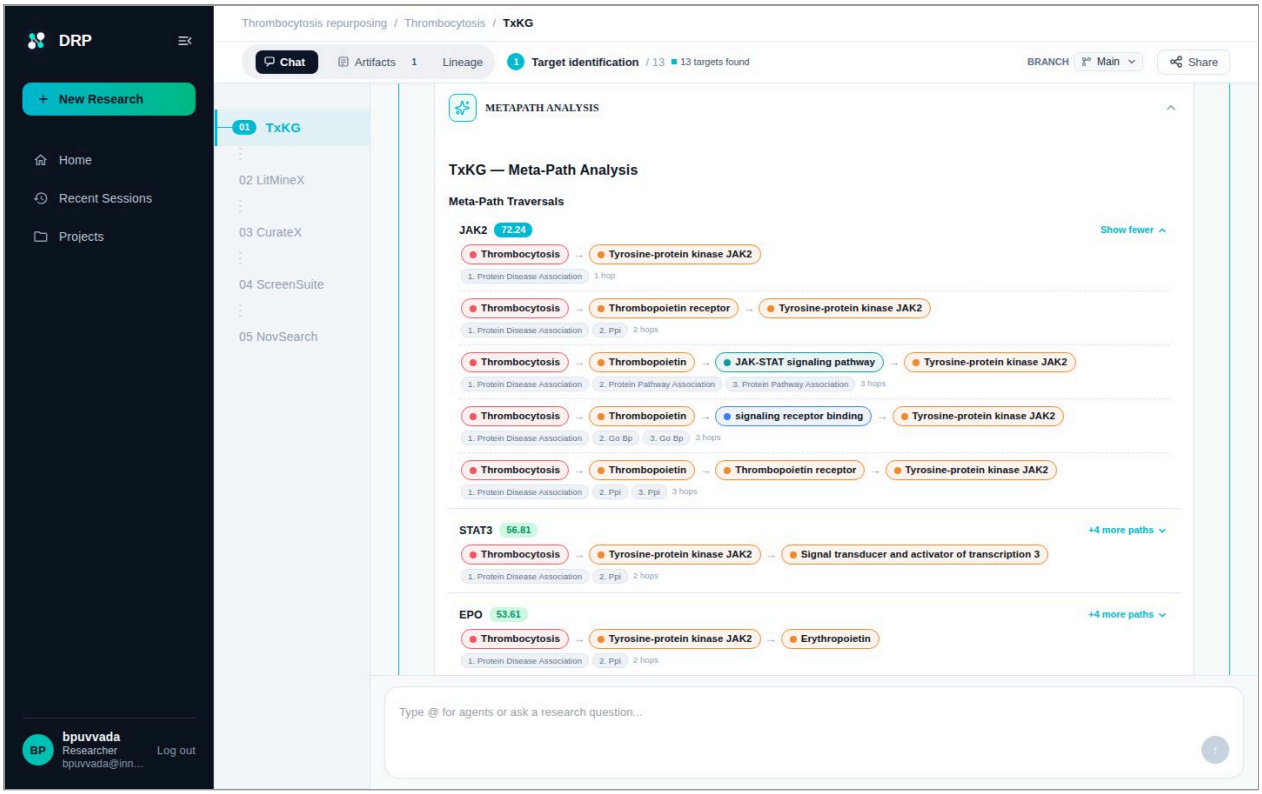


Figure 3.6.4 Meta-path analysis: why each target links to the disease.

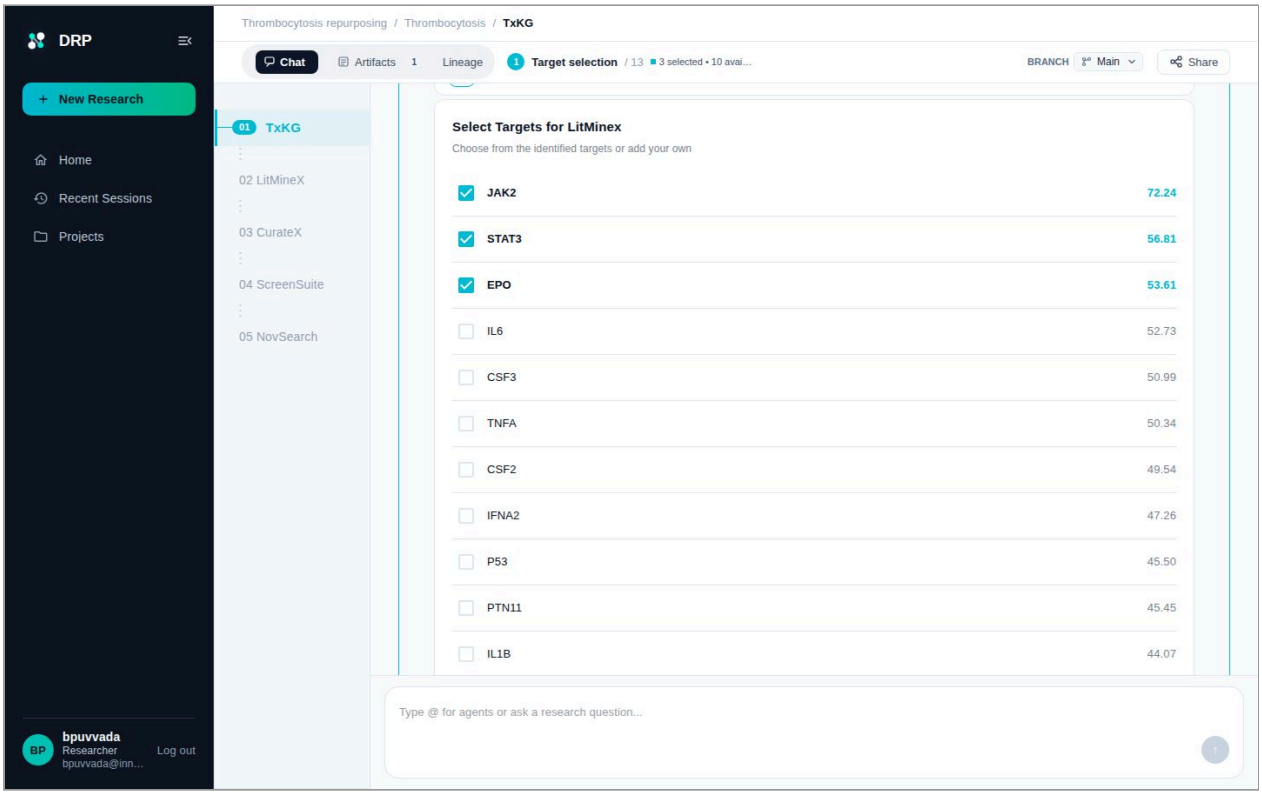


Figure 3.6.5 Select Targets for LitMineX (top three pre-ticked; custom targets can be added).

3.7 Step 2 · LitMineX: literature mining

Subject Area	**Step 2 · LitMineX: literature mining**	**Topic**	Agent
Input	The selected targets (gene symbols).	**Flow ID**	F07
What happens	The LitMineX job searches PubMed and scores each article (about 30 seconds). Results: `GET /agents/litminex/{jobId}/results` and `/insights`.		
Output	Ranked articles with year, confidence and keywords; a hover preview; the Article Detail panel (abstract, PubMed Central link, **Ask about this paper**, **Save Article**).		
Next	**Continue to CurateX**, choosing one protein target.		

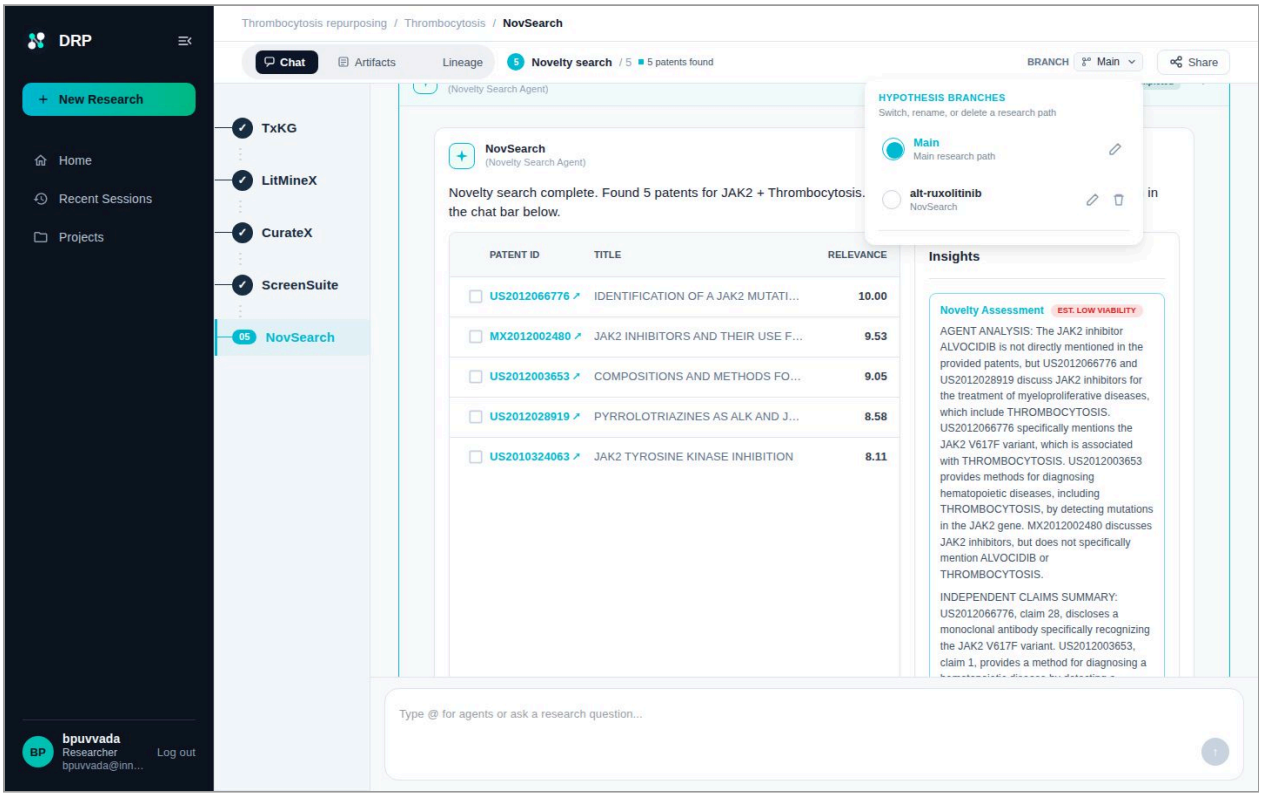


Figure 3.7.1 LitMineX results and the Article Relevance insight.

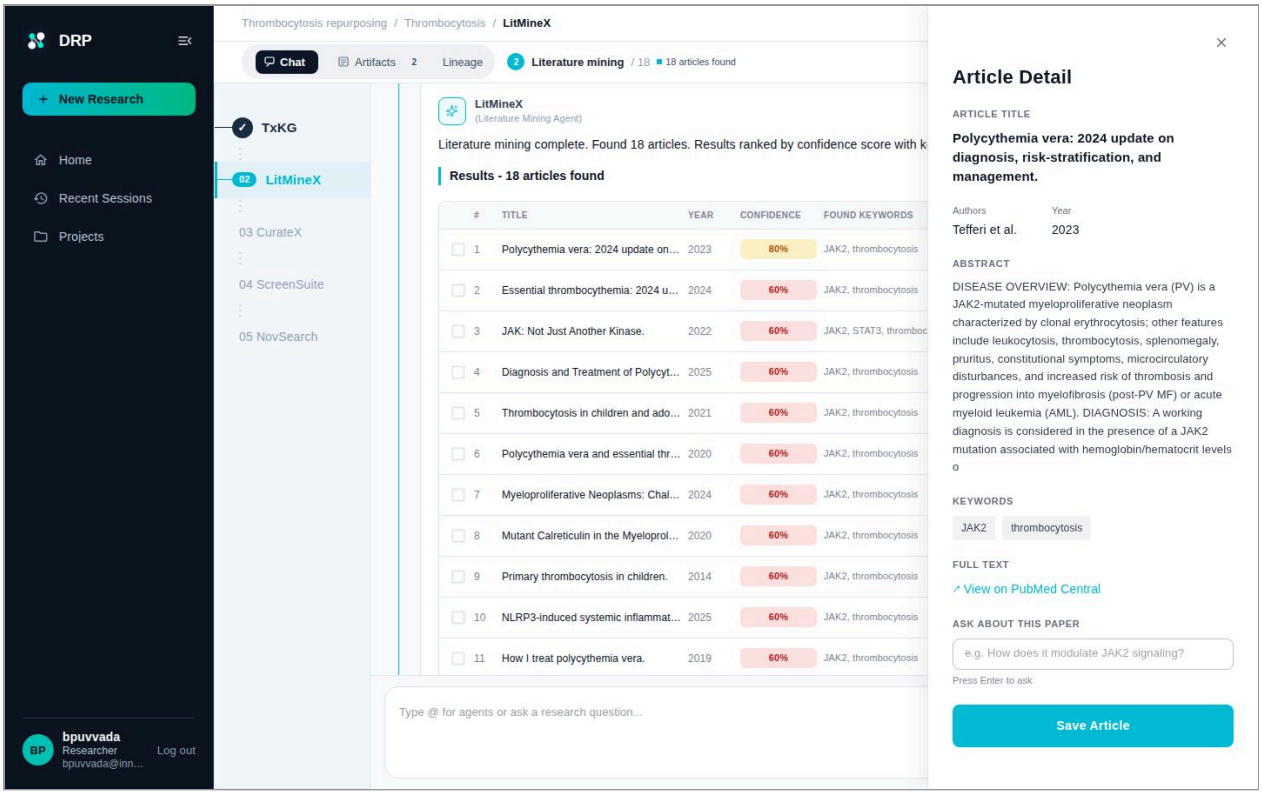


Figure 3.7.2 Article Detail panel.

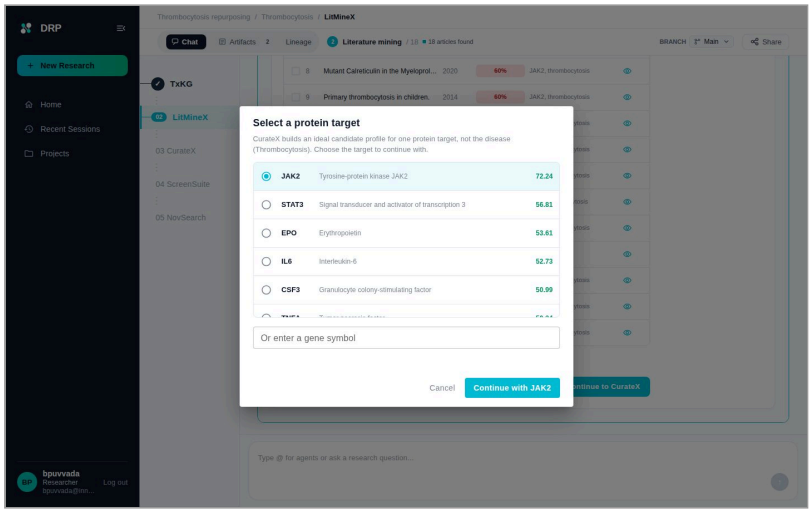


Figure 3.7.3 Select the protein target for CurateX.

3.8 Step 3 · CurateX: drug curation

Subject Area	Step 3 · CurateX: drug curation	Topic	Agent
Input	One protein target.	**Flow ID**	F08
What happens	Stage 1 builds an Ideal Candidate Profile from the target's known ligands (`GET /agents/curatex/{jobId}/profile`). The researcher can edit values and weights (total 100) or add parameters. **Submit Profile** starts stage 2 (`POST /agents/curatex/compounds`), which scores existing drugs against the profile.		
Output	Ranked candidates with score, matched and mismatched criteria, per-criterion Match Details (two decimals) and a written recommendation.		
Next	**screen these in ScreenSuite** → (or **Next**) opens the screening picker.		

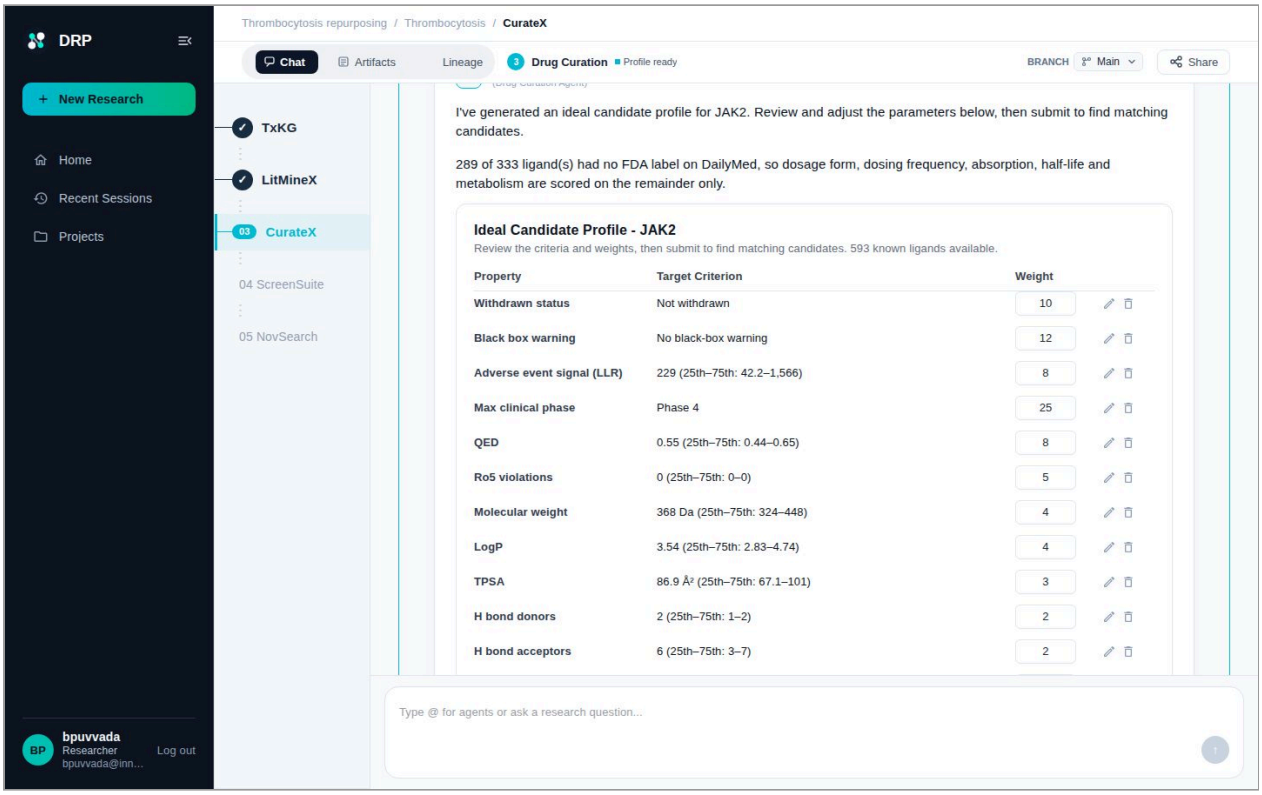


Figure 3.8.1 Ideal Candidate Profile for JAK2.

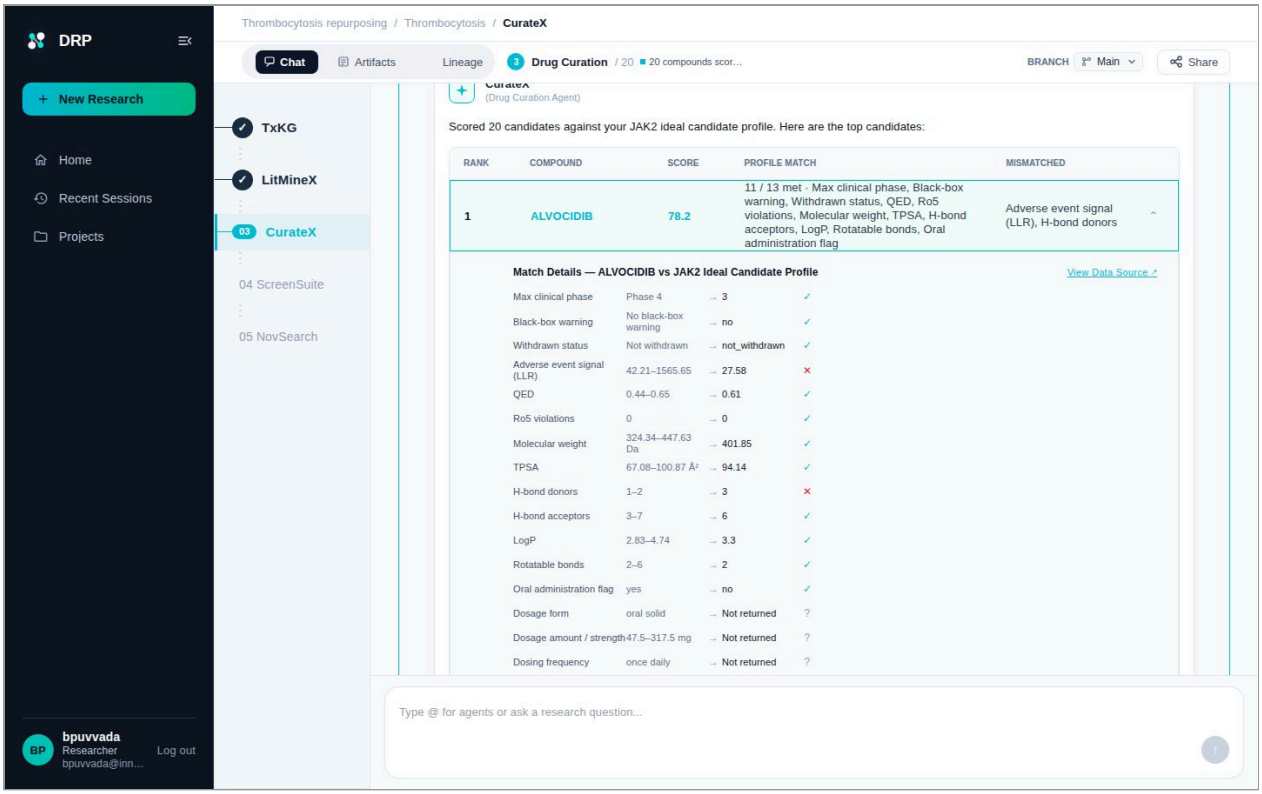


Figure 3.8.2 Scored candidates, with match details for the top compound.

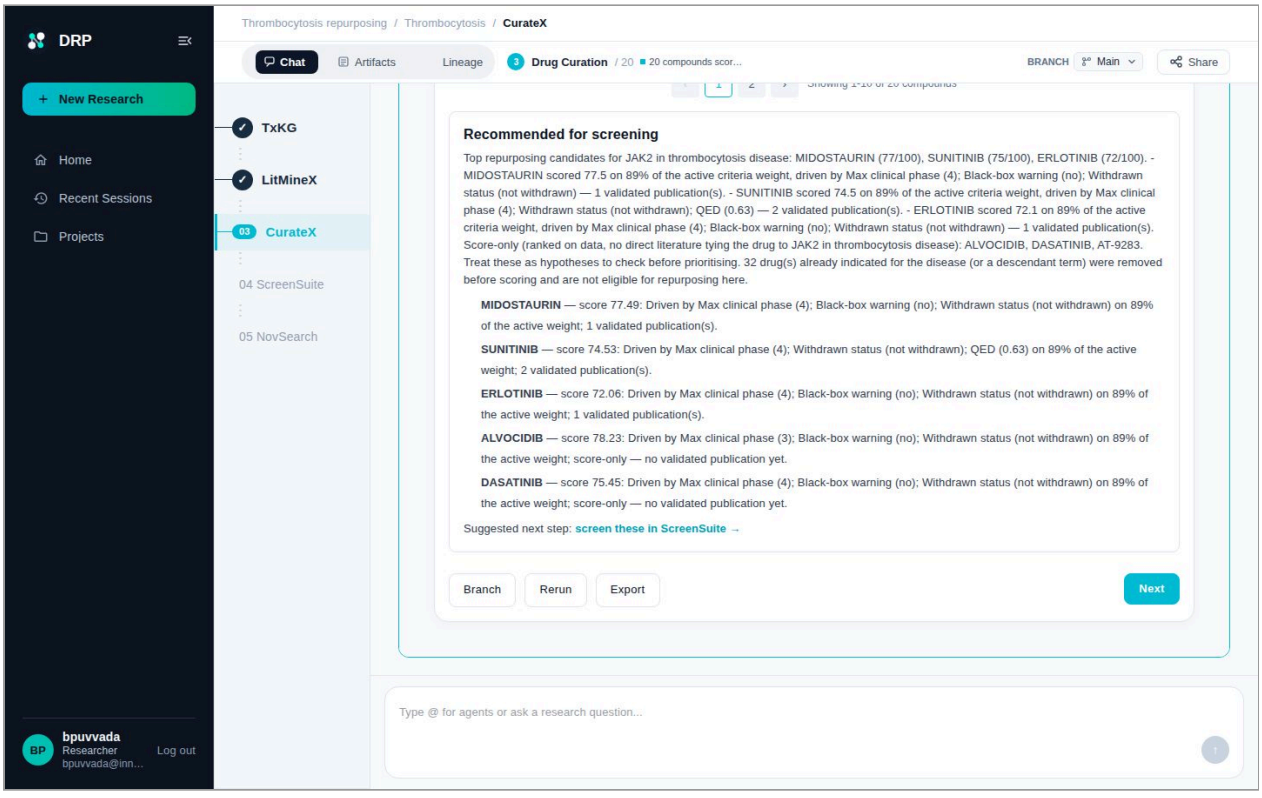


Figure 3.8.3 Recommended for screening.

3.9 Step 4 · ScreenSuite: virtual screening

Subject Area	**Step 4 · ScreenSuite: virtual screening**	**Topic**	Agent
Input	Proteins × candidates chosen in the picker; then a PDB structure per protein when several match.	**Flow ID**	F09
What happens	`POST /sessions/{id}/steps` starts docking. If a structure must be chosen, the card shows **Needs input** and a structure picker. Docking runs first; ProLIF interaction profiling follows. Results: `GET /agents/screensuite/{jobId}/results`, interactions per result, files per result and for the whole screening.		
Output	Docking table (protein, mode, binding affinity in kcal/mol, PDB, ligand) with **INTERACTION PROFILE**; the backend's recommendation; a Mol* 3D view showing every ProLIF interaction as a dashed line; the interaction list (select one to zoom to it); downloads.		
Next	**Continue to NovSearch**: pick a docked combination or type a query.		

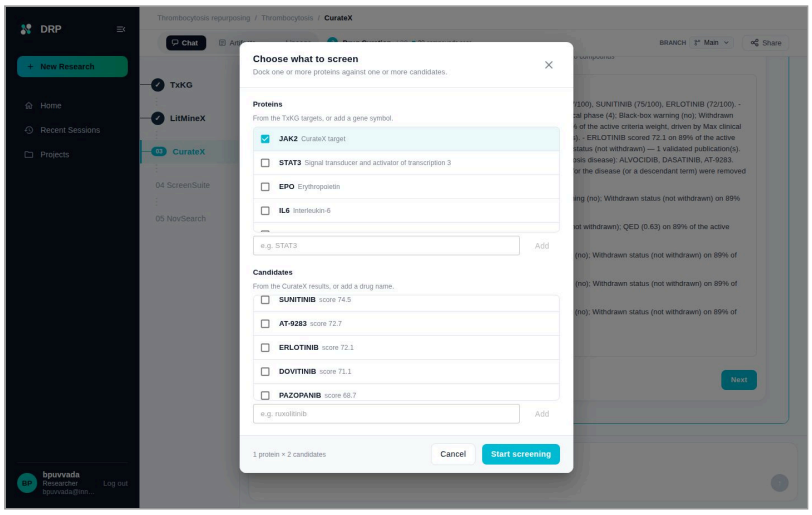


Figure 3.9.1 Choose what to screen.

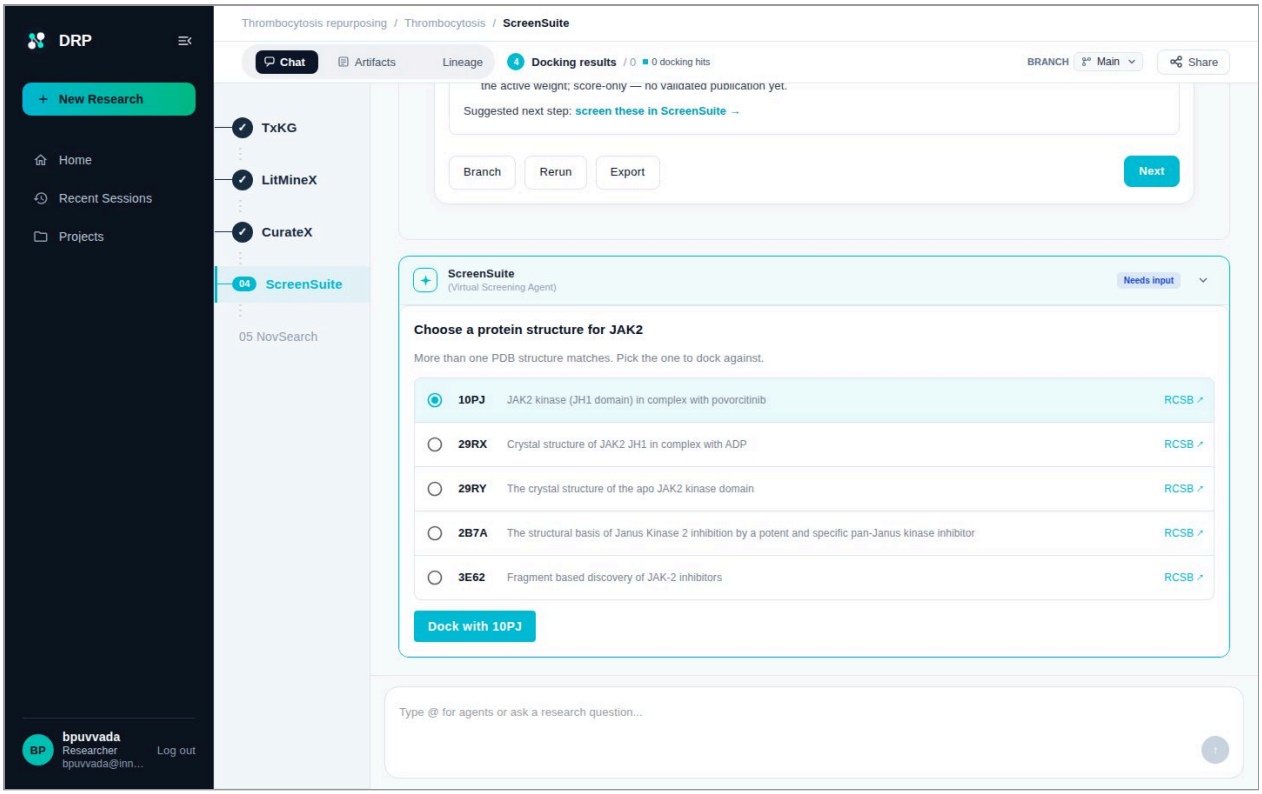


Figure 3.9.2 Choose a protein structure (card shows Needs input).

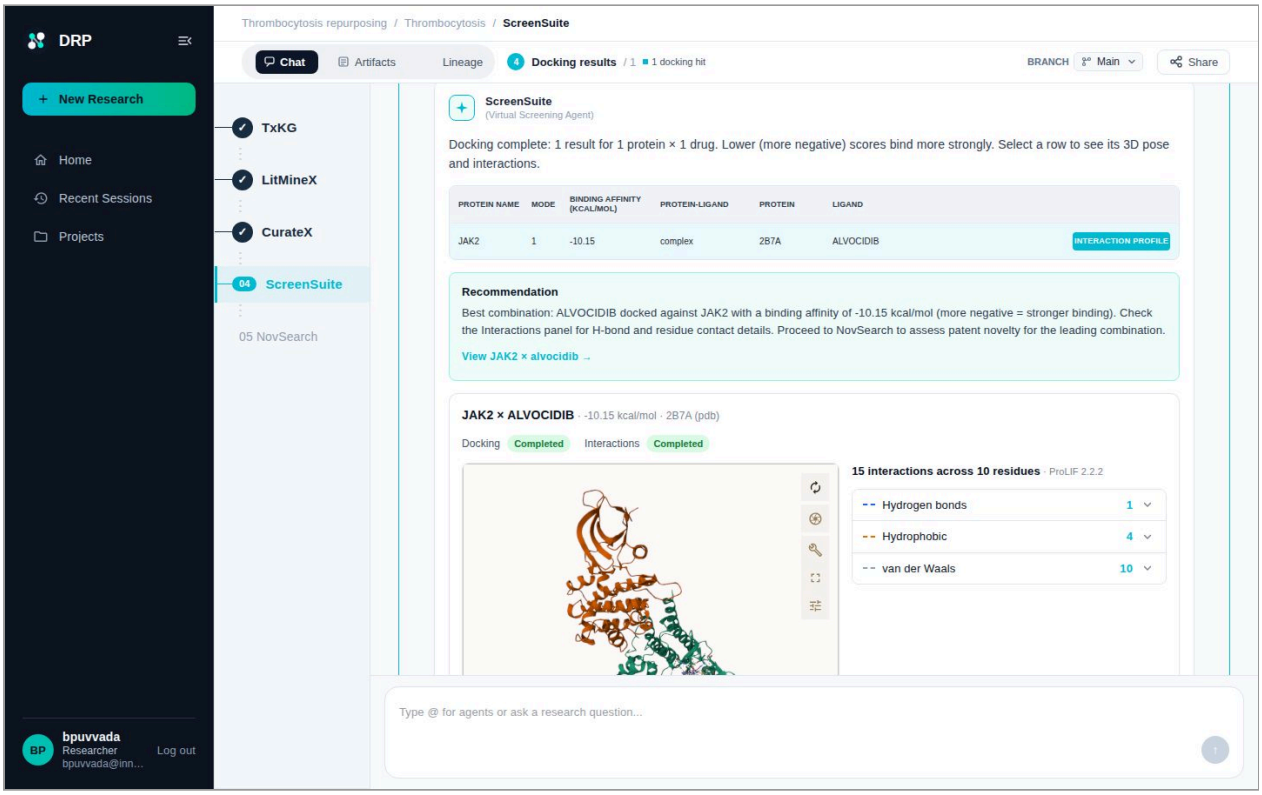


Figure 3.9.3 Docking results and recommendation.

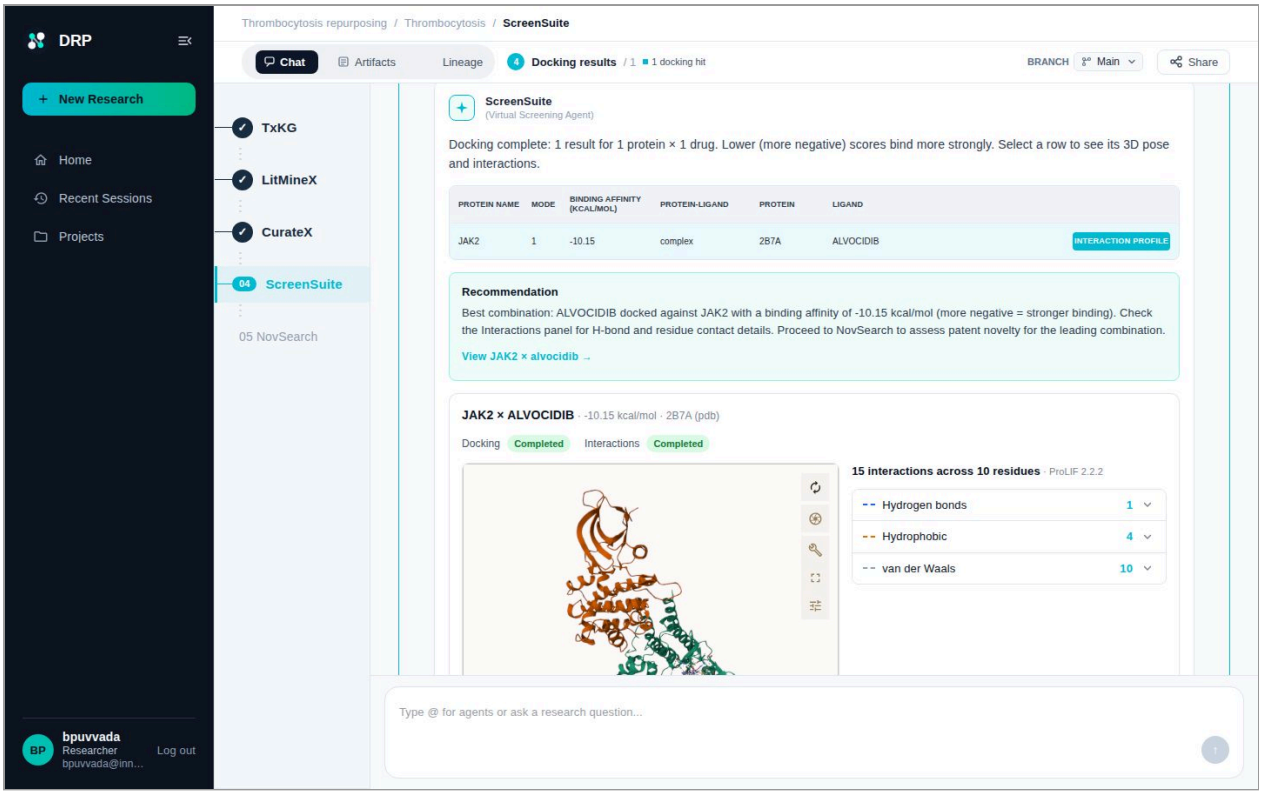


Figure 3.9.4 3D view with ProLIF interactions, the interaction list and downloads.

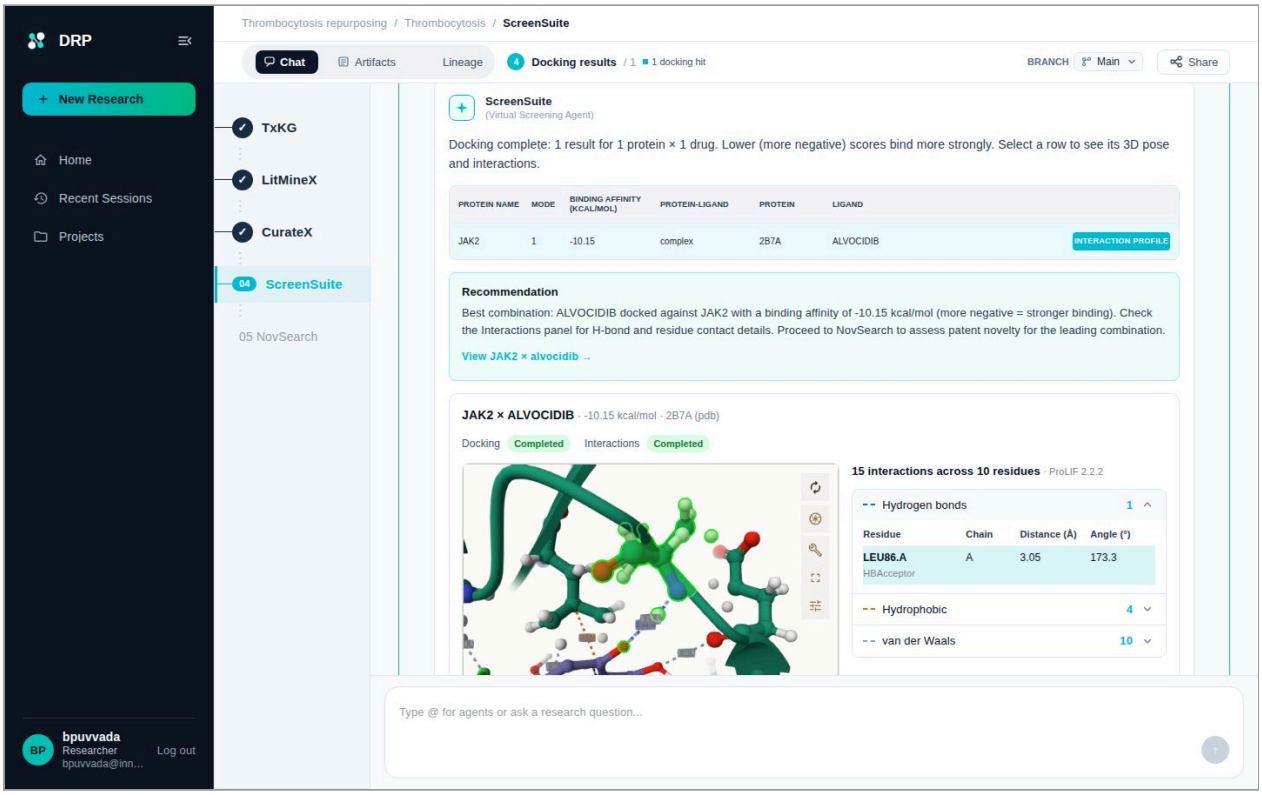


Figure 3.9.5 One interaction selected: residue and ligand atom highlighted, camera zoomed.

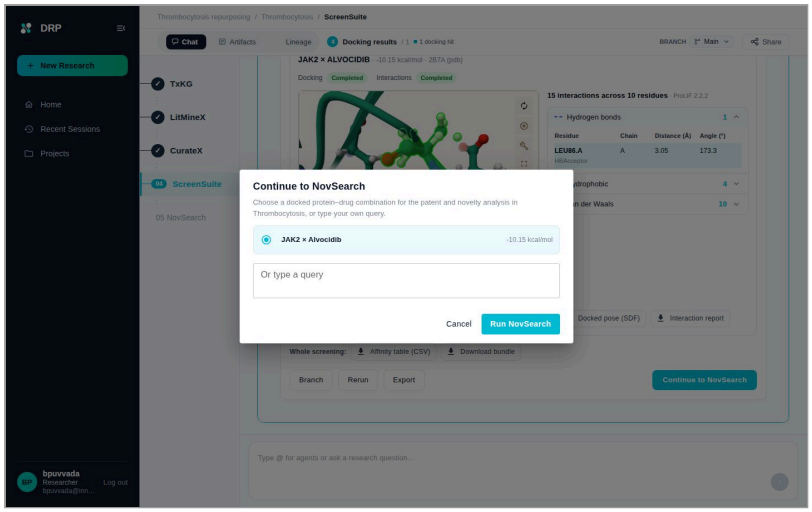


Figure 3.9.6 Continue to NovSearch.

3.10 Step 5 · NovSearch: novelty search

Subject Area	**Step 5 · NovSearch: novelty search**	**Topic**	Agent
Input	A protein–drug combination (or a typed query).	**Flow ID**	F10
What happens	The NovSearch job finds relevant patents and assesses novelty (`GET /agents/novsearch/{jobId}/report`). Ticking patents and asking in the chat bar sends `POST /agents/novsearch/ask` scoped to that run.		
Output	Patents with relevance and links; the Novelty Assessment (claims, novelty, freedom-to-operate risk, recommendations); answers to patent questions.		
Next	**Finish Research** → **Continue Research** or **End Task** (saves the session); the summary offers **Export Report**.		

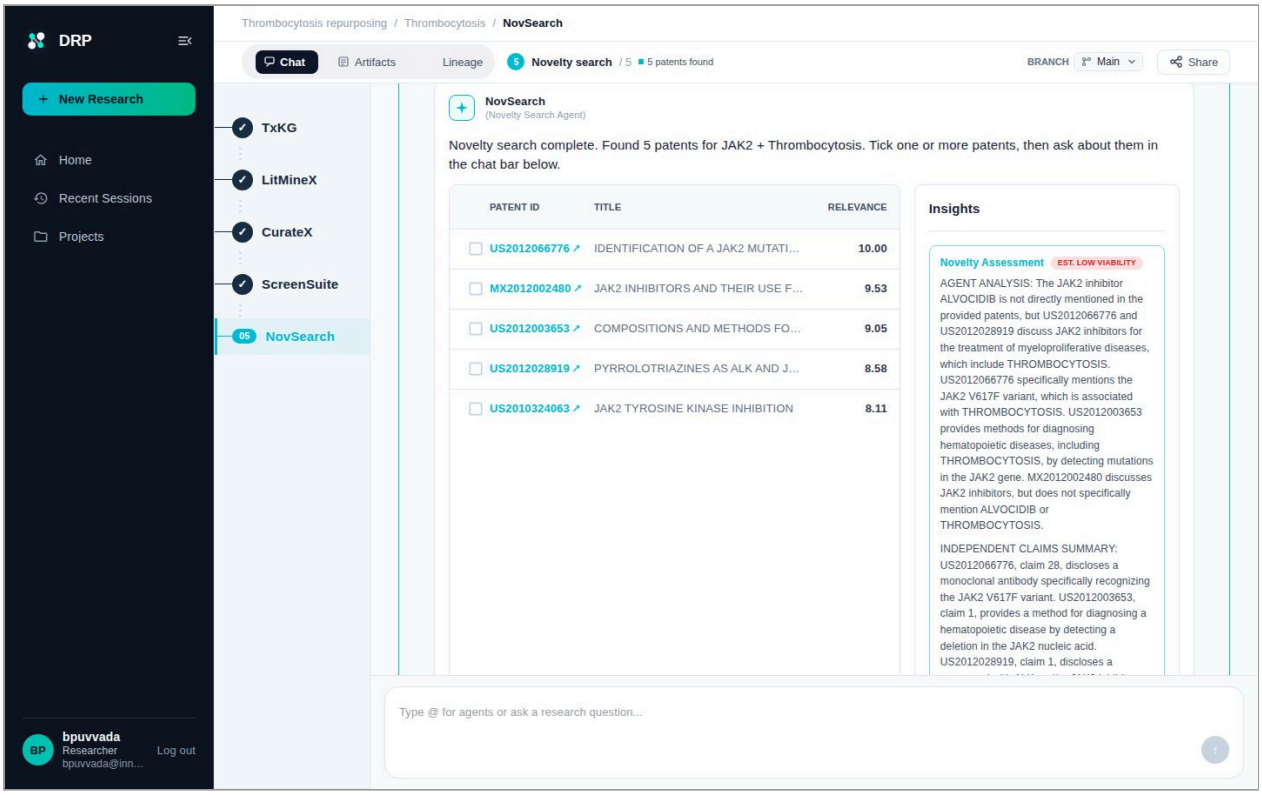


Figure 3.10.1 NovSearch results and Novelty Assessment.

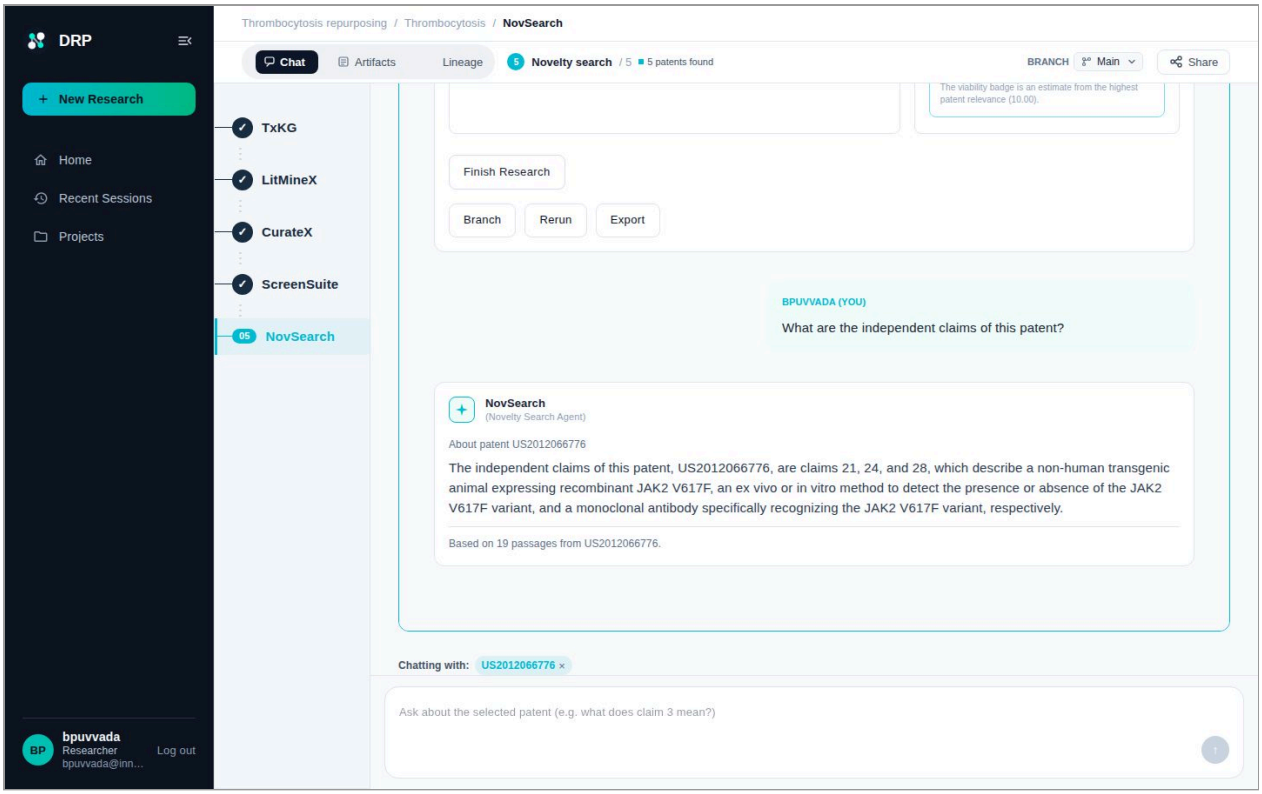


Figure 3.10.2 A question about the ticked patents, answered in the thread.

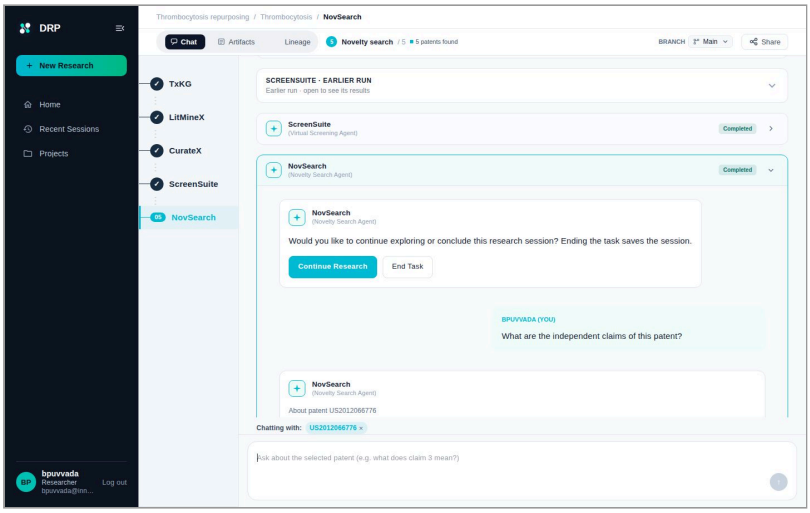


Figure 3.10.3 Finish Research: continue or end the task.

3.11 Branch, rerun and export

Subject Area	Branch, rerun and export	Topic	Session
Branch	**Branch** on any card: name, step to branch from, description. Nothing re-runs; the next run on the branch starts from that point. The **BRANCH** menu switches between Main and branches, and branches are restored when the session is reopened.	**Flow ID**	F11
Rerun	Runs the same step again; the earlier run stays above as "Earlier run" and can be opened to see its results.		
Export	CSV (`POST /exports`) or PDF report (`POST /exports/pdf`) for a card.		

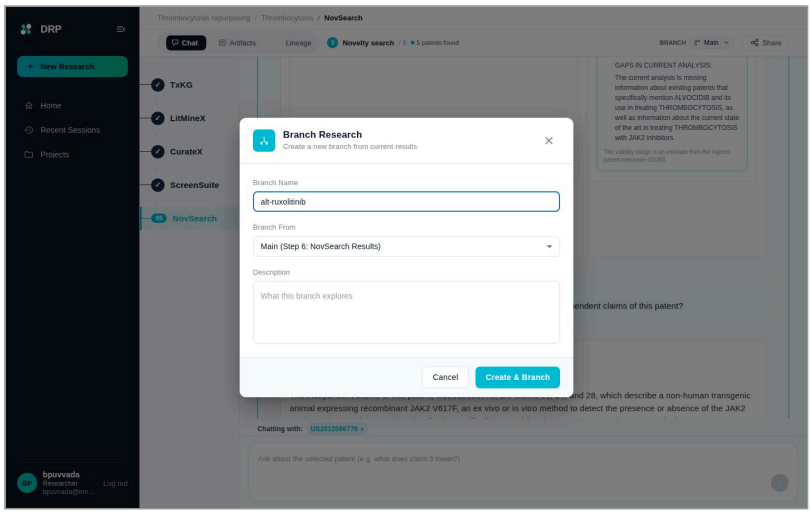


Figure 3.11.1 Branch Research dialog.

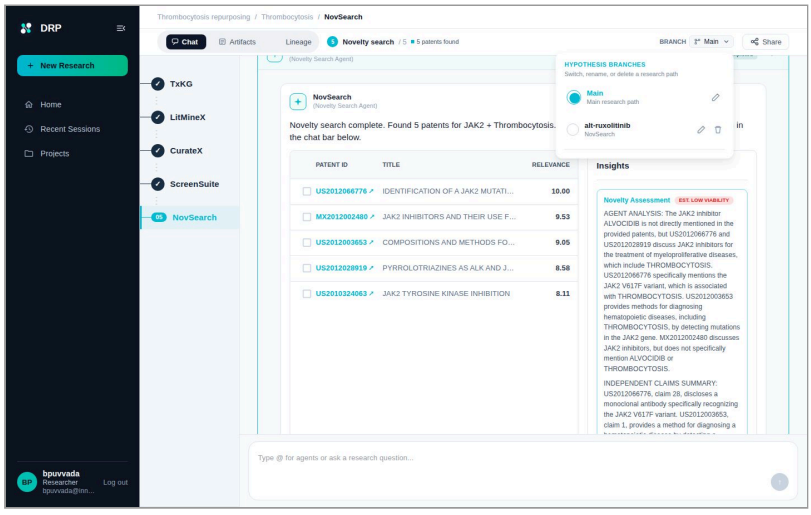


Figure 3.11.2 BRANCH menu with Main and a branch.

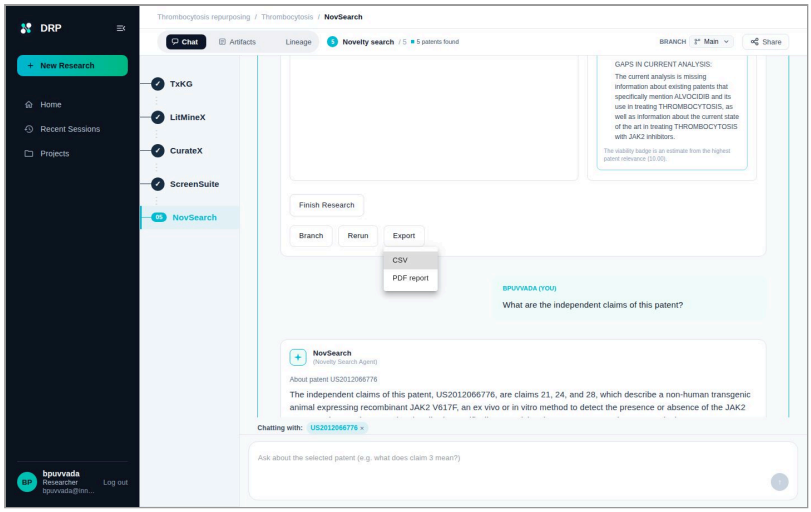


Figure 3.11.3 Export menu.

3.12 Artifacts and lineage

Subject Area	Artifacts and lineage	Topic	Session
Artifacts	Every saved result of the session (`GET /sessions/{id}/artifacts`), with **Add to project**.	**Flow ID**	F12
Lineage	"How you got here": the branch and, per step, its status and selection.		

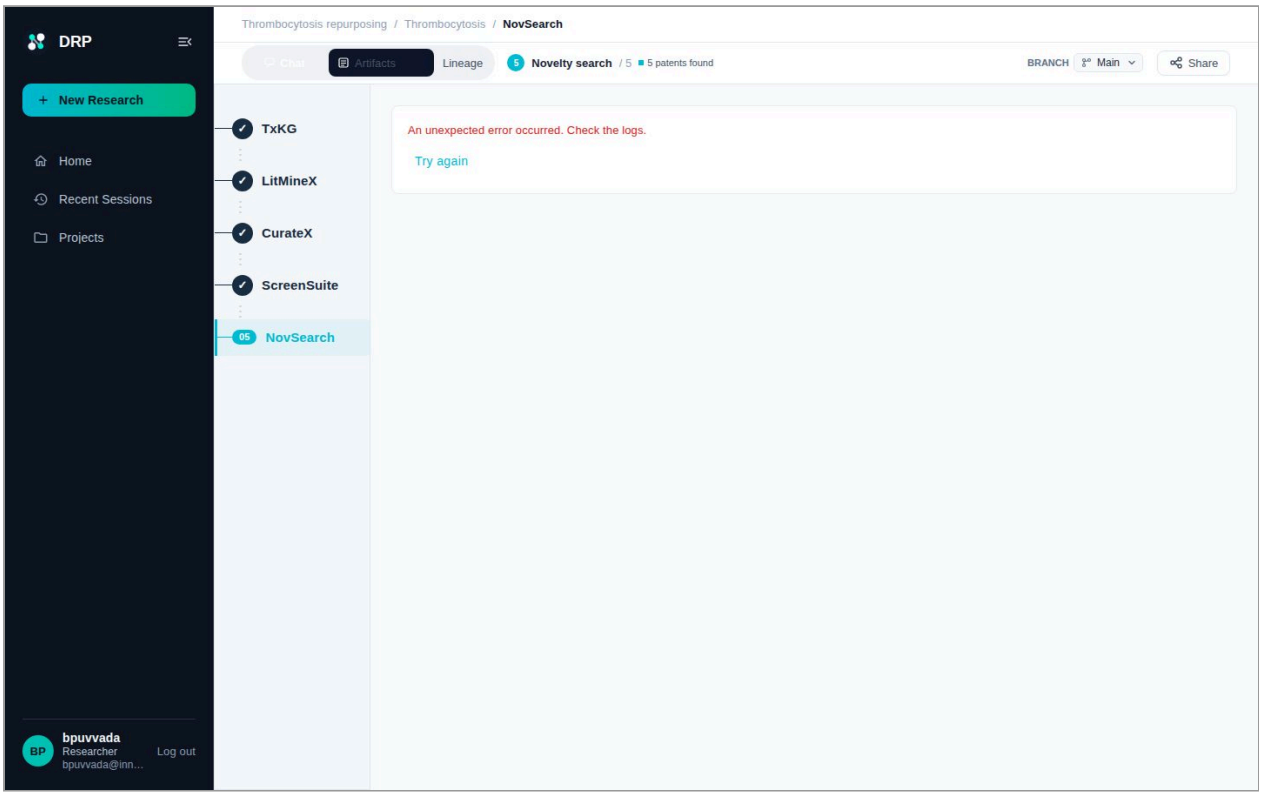


Figure 3.12.1 Artifacts tab.

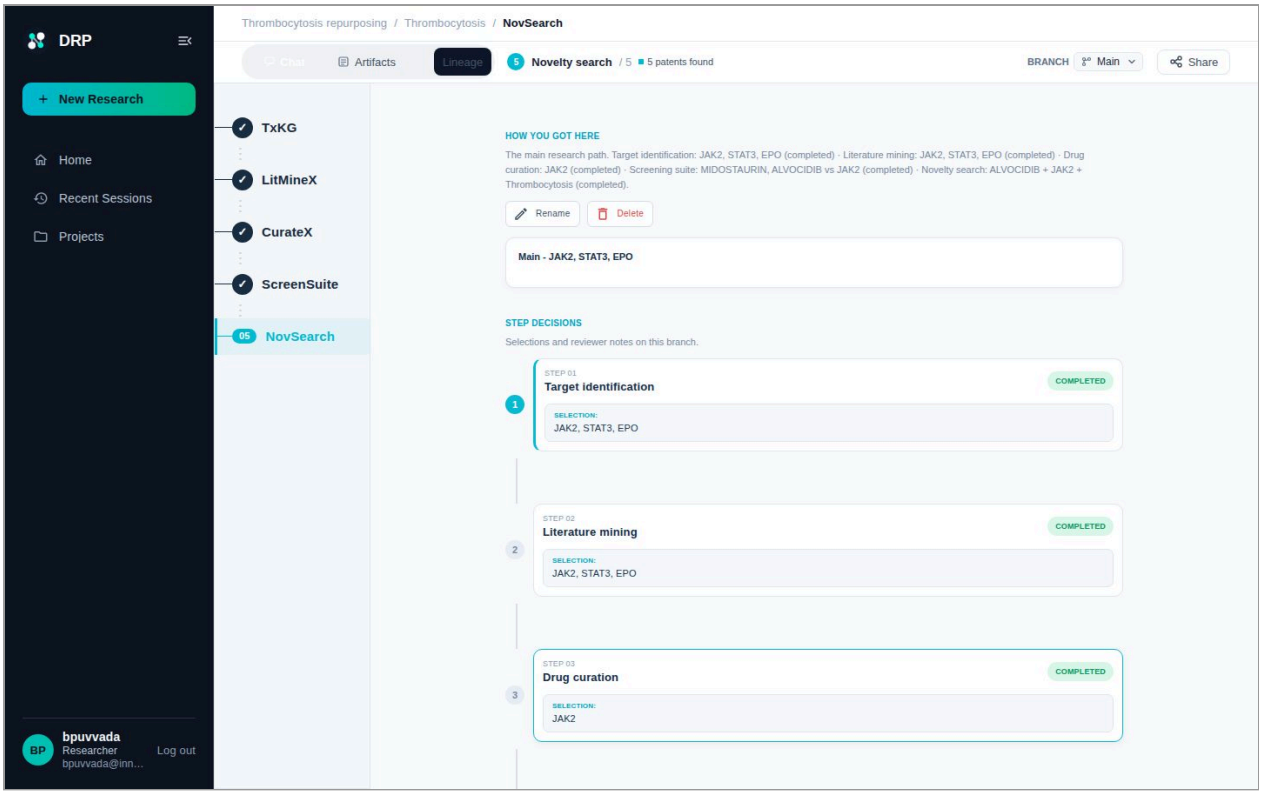


Figure 3.12.2 Lineage tab.

3.13 Share

Subject Area	Share	Topic	Session
Purpose	Share the session.	**Flow ID**	F13
What happens	**Copy link** copies the session's URL (it reopens the session for the signed-in owner). Email invitations are disabled until a backend sharing service exists; the dialog says so.		

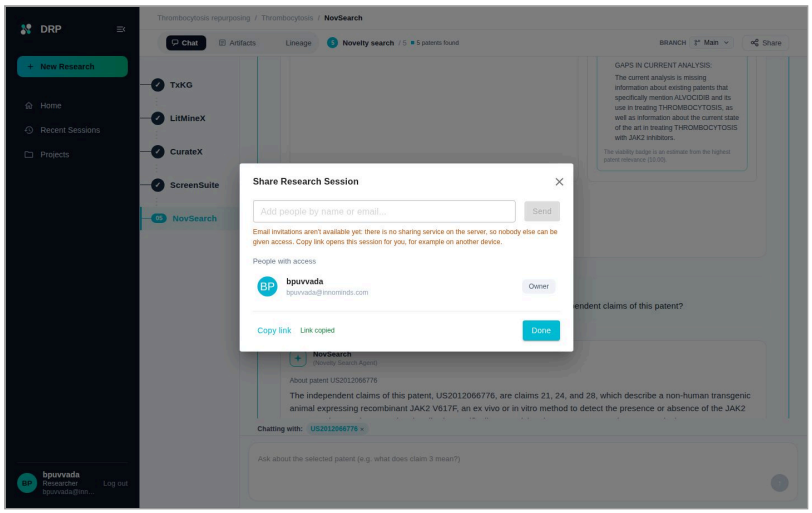


Figure 3.13.1 Share Research Session.

3.14 Recent sessions and project page

Subject Area	Recent sessions and project page	Topic	Organise
Recent Sessions	All sessions, newest first, with module, status, project and update time (UTC); search; filter by module and status; open, **Add to project** (folder icon) or delete.	**Flow ID**	F14
Project page	The sessions in the project and the results filed into it, grouped by type.		

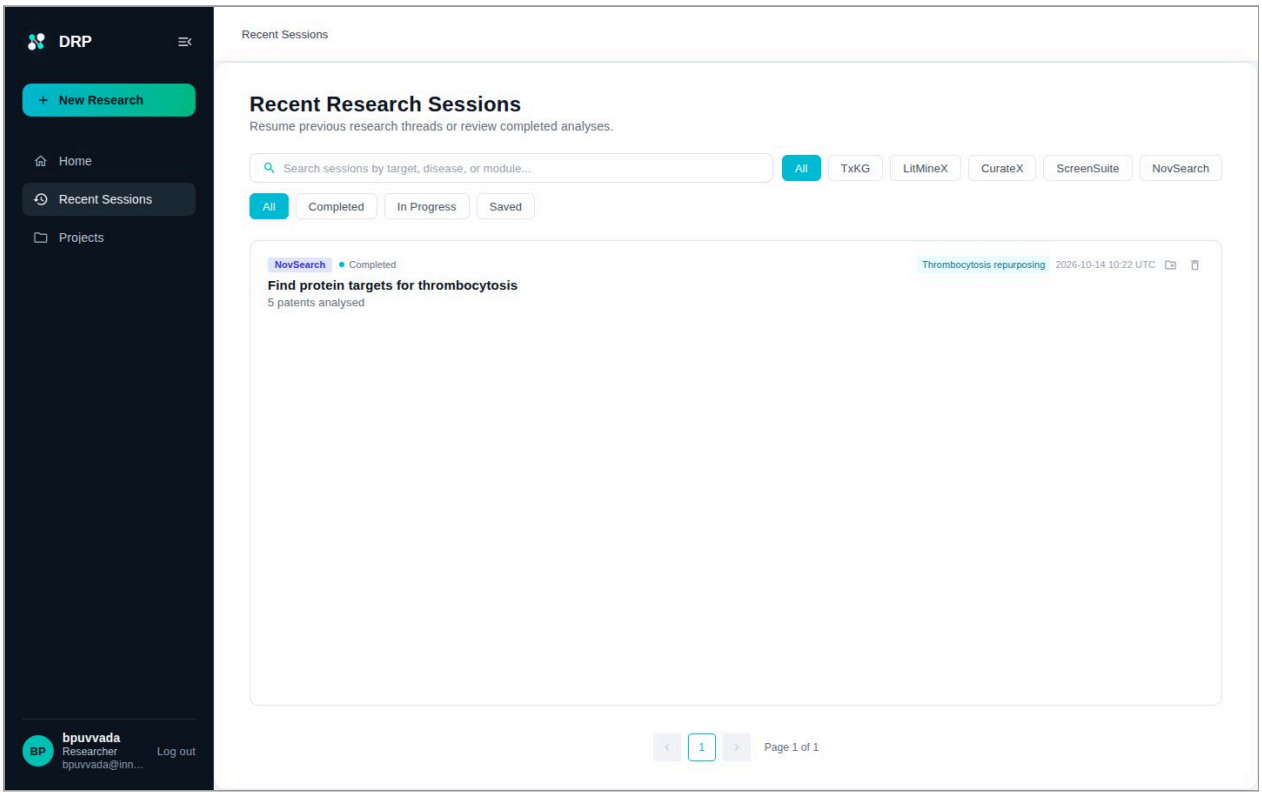


Figure 3.14.1 Recent Sessions, with the project chip and Add to project.

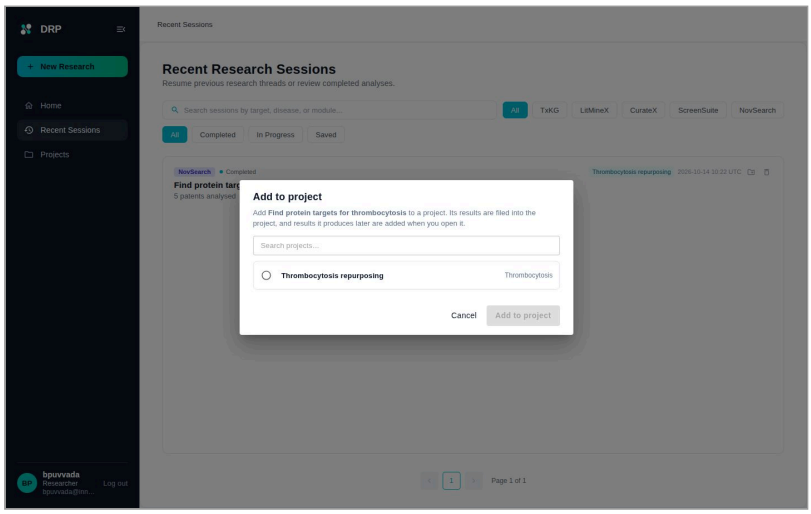


Figure 3.14.2 Add an existing session to a project.

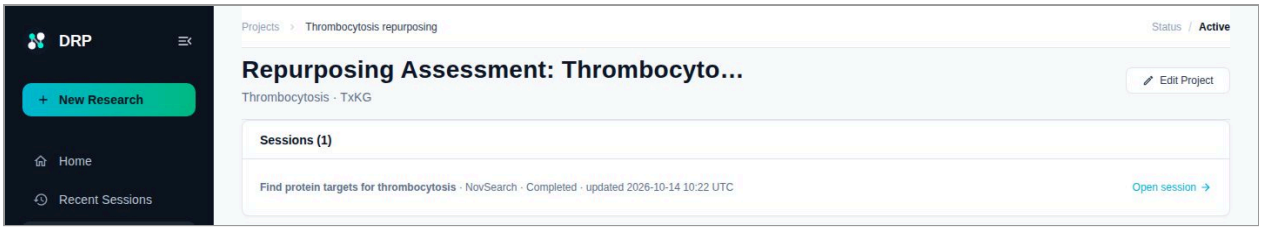


Figure 3.14.3 Project page with its Sessions section (top of page).

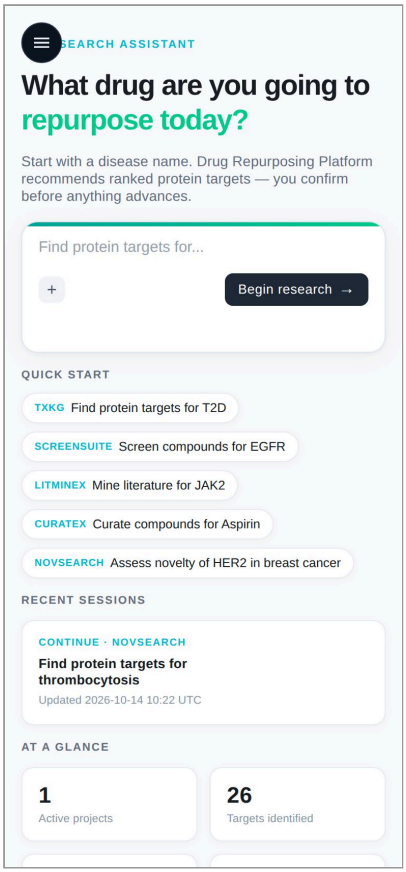


Figure 3.14.4 Home at phone width (390 px).

4. Current Limitations

Area	Limitation
Projects	The backend does not yet store a session's project; the link is kept in the browser (C07).
Patent Q&A	Patent questions and answers are kept in the browser until the backend stores them.
Share	Email invitations need a backend sharing service; Copy link works for the owner.
Help / Privacy / Terms	Pages exist; approved text is still to be provided.
Development port	Run locally on port 3000 only (C05).
Project results	On 2026-10-14 the backend returned HTTP 500 for GET /projects/{id}/items on the walkthrough project, so its saved-results list could not be shown (reported to the backend).
ScreenSuite	When several drugs are screened, the backend can return results for only some of them without listing the rest as failures (reported to the backend).